📘 Course Outline: PHP, MySQL, JavaScript & Node.js

Week 1 – Foundations

- Web development overview (frontend vs backend)
- Setting up environment: PHP, MySQL, Node.js, npm, VS Code
- Client–server architecture
- First scripts in PHP and JavaScript

Week 2 – PHP Basics

- Variables, data types, operators
- Control structures (if, switch, loops)
- Functions and scope
- Handling HTML forms with PHP

Week 3 – PHP Arrays & Strings

- Indexed, associative, and multidimensional arrays
- Array functions (`in_array`, `array_merge`, `array_filter`)
- String functions (`strlen`, `substr`, `strtoupper`)
- Practical: student grade manager

Week 4 – PHP & MySQL Integration

- Introduction to databases
- Connecting PHP to MySQL (`mysqli`, `PDO`)
- CRUD operations (Create, Read, Update, Delete)
- Practical: simple user registration system

Week 5 – PHP Advanced Topics

- Sessions and cookies
- File handling (`file_get_contents`, `file_put_contents`)
- Error handling
- Building a mini blog with PHP + MySQL

Week 6 – JavaScript Fundamentals

- Variables (`let`, `const`, `var`)
- Functions and scope
- DOM manipulation (`getElementById`, `querySelector`)
- Events (`onclick`, `addEventListener`)
- Practical: interactive form validation

Week 7 – JavaScript Arrays, Objects & JSON

- Array methods (`map`, `filter`, `reduce`)
- Objects and JSON
- Loops (`for`, `for…in`, `for…of`)
- Practical: to-do list app

Week 8 – JavaScript Advanced Topics

- ES6+ features (arrow functions, template literals, destructuring)
- Asynchronous JavaScript (callbacks, promises, async/await)
- Fetch API for HTTP requests
- Practical: consuming a public API (weather, quotes)

Week 9 – Node.js Basics

- Introduction to Node.js runtime
- npm and package management
- Creating a simple server with `http` module
- Understanding modules (`require`, `import/export`)

Week 10 – Node.js with Express

- Installing and using Express.js
- Routing (GET, POST, PUT, DELETE)
- Middleware
- Serving static files
- Practical: REST API for products

Week 11 – Node.js & Database Integration

- Connecting Node.js with MySQL (or MongoDB)

- CRUD operations
- Using an ORM (Sequelize / Mongoose)
- Practical: student record management API

Week 12 – Capstone Project

- Full-stack project combining:
  - PHP + MySQL for admin panel or legacy backend
  - JavaScript for frontend interactivity
  - Node.js + Express for modern API backend
- Example project: **E-commerce mini app** or **Student Portal**

✅ Key Takeaways

- **PHP + MySQL**
  : classic server-side scripting and database integration
- **JavaScript**
  : frontend interactivity and async programming
- **Node.js**
  : modern backend API development with Express

PHP: Hypertext Preprocessor



PHP: Hypertext Preprocessor

1. Introduction and Background

PHP is an open source server-side scripting language designed for Web development to produce dynamic Web pages. It is one of the first developed server-side scripting languages to be embedded into an HTML source document rather than calling an external file to process data. The code is interpreted by a Web server with a PHP processor module which generates the resulting Web page. PHP can be deployed on most Web servers and also as a standalone shell on almost every operating system and platform, free of charge. PHP was a competitor to Microsoft's Active Server Pages (ASP) server-side script engine and similar languages, but gradually received better acceptance than ASP and by now PHP is installed on more than 20 million Web sites and 1 million Web servers. Notable software that uses PHP includes Drupal, Joomla, MediaWiki, and WordPress.

PHP was originally created by Rasmus Lerdorf in 1995. The main implementation of PHP is now produced by The PHP Group and serves as the formal reference to the PHP language. PHP is free software released under the PHP License, which is incompatible with the GNU General Public License (GPL) due to restrictions on the usage of the term PHP.

While PHP originally stood for Personal Home Page, it is now said to stand for PHP: Hypertext Preprocessor, a recursive acronym.

2. Dynamic web sites

The static model of the web interaction is based on a set of pre-developed static web pages stored on a host server, and in 3 basic steps:

1. A client sends a request for a web page to the host.
2. The host sends a copy of the requested page to the client.
3. If desired, points 2 and 3 are repeated for new pages.

A node in the web which manages the host tasks is called a web server. In the static model, Figure 2.1, the host has no ability to analyze the request and adjust the response accordingly. The response is a requested pre-designed web page. The request-response

exchange is therefore called static. However, the exchange protocol used, HTTP, provides possibilities for some additional items of information to be sent to the host with the request without any instructions from the client. In the same way, the responding host can include additional information with the response, usually hidden for the receiver. The host has also capabilities to forward messages to other programs beyond the web server for additional processing. These possibilities for information processing behind the scene make it possible to create the additional functionality.

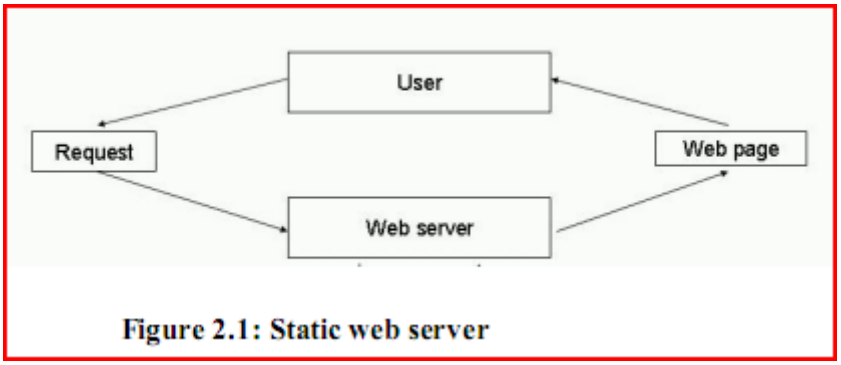


Figure 2.1: Static web server

We shall use the term dynamic web site to emphasize that we are not concerned with a simple set of web pages with HTML tags, but with applications in which the pages returned to the client can be dynamically adjusted to fit the individual requests of the client.

Important characteristics of a dynamic web site are the ability to authenticate you, i.e. to verify your identity, to record your performance history, to react on the time for the request, to keep track of your interactions from you start a session and to its end, and sometimes even from session to session. The dynamic web site can be summarized by Figure 2.2.

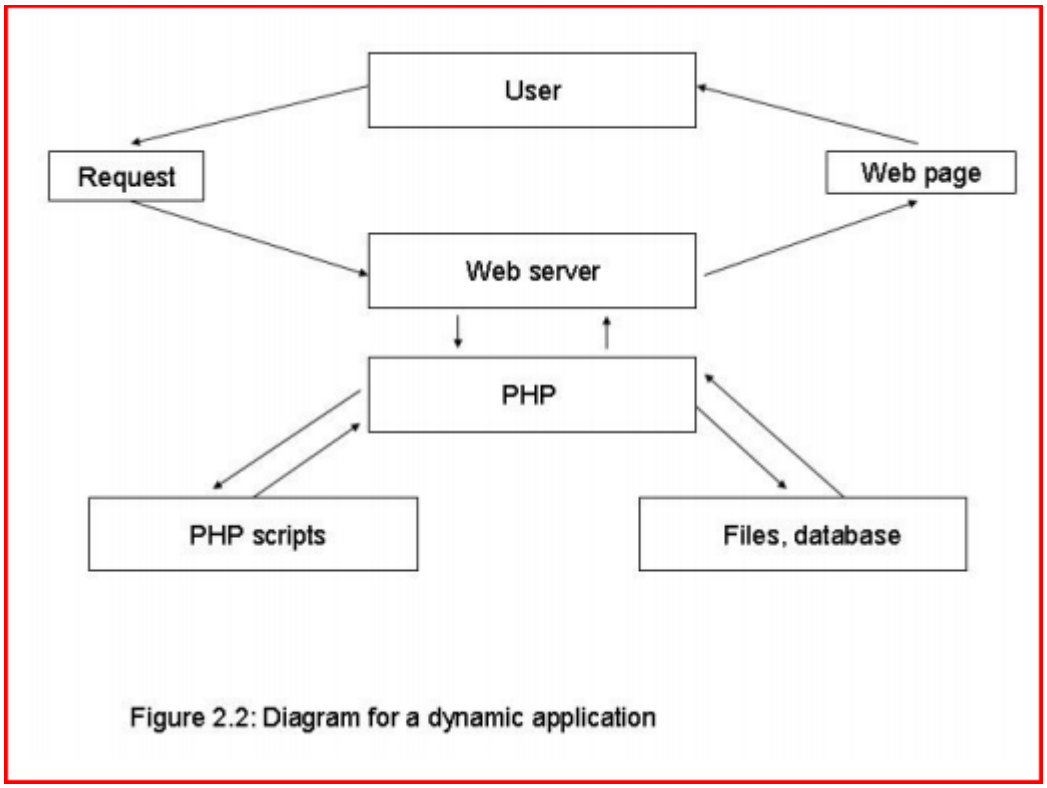


Figure 2.2: Diagram for a dynamic application

3. CGI and PERL

The first step toward dynamic web pages is the possibility for a remote client to request the execution of a process at the host. Use of the FORM tags of HTML requires, for example, that the server can perform a processing of the data submitted on the form. A program must exist for this purpose at the host site, and the web server must be able to communicate with this. We shall refer to such a program which supplements the HTML pages as a script.

The Common Gateway Interface, CGI, is a protocol specifying how certain scripts can communicate with web servers. One of the most frequently used tools for creating such scripts is the script language PERL. A PERL script stored in the host computer can be supplied with data from a request, for example sent by a HTML FORM page. The script can be designed to perform a variety of tasks such as save and retrieve data from a database, update a log, keep track of visitors, run a course, etc. It can also be designed to perform its task and then leave the result to the web server, which returns a web page generated by means of the script to the requesting client. Programming languages such as C, C++, C# and JAVA can also be used for creating scripts. One reason for the popularity of PERL is that scripts programmed in PERL can be ported from one operating system to another with no or little modification.

3.1. Applications Program Interfaces

A PERL-CGI application is time-consuming because PERL scripts must be loaded, executed and unloaded each time they are used like interpretive programs, and do not offer the flexibility which may be required.

To improve this situation, Application Servers were developed. An application server is a service operating behind the web server. It processes script code, which the web server does not understand, and returns the results to the web server for returning to the requesting client. The applications server is a resource of permanently loaded executable programs, and is referred to as an Applications Program Interface, API. The advantages of using an API compared with the earlier interpretative programs are increased speed and flexibility because no loading and interpretation is needed. The disadvantage is that the API programs must be implemented and compiled separately for each type of operating system, and requires more memory space.

3.2. PHP Language

The well-known API tools include the ASP and ASP.NET from Microsoft, the open source system PHP, iHTML from Inline Internet Systems, and ColdFusion MX from Macromedia. In this course, we are leaving the comparisons between the tools to evaluators and sales people, and concentrate on PHP because it is an open source tool, easily available and supported by a large community of users. PHP was introduced in 1995 as Personal Home Pages. Since then, PHP has been developed to a very powerful tool for treating dynamic web sites.

The language by which we design our scripts is the PHP Language. Files in which scripts are saved are recognized by their extensions, .**php**. In the previous paragraph, the advantage of using a web API instead of an interpretive approach was emphasized. It started out as a scripting language based on CGI. Later, the API was developed. The current version is PHP 5 which is a powerful system with an embedded database system, **SQLite**. Be certain that you have the PHP 5 version installed.

PHP is widely used by individuals and enterprises among which there exist an active interchange of software and experience.

4. PHP TAGS

The PHP interpreter only executes PHP code within its delimiters. Anything outside its delimiters is not processed by PHP (although non-PHP text is still subject to control structures described in PHP code). The most common delimiters are **<?php** to open and **?>** to close PHP sections. **<script language="php">** and **</script>** delimiters are also available, as are the shortened forms **<?** or **<?=** (which is used to echo back a string or variable) and **?>** as well as ASP-style short forms **<%** or **<%=** and **%>.**

In terms of keywords and language syntax, PHP is similar to most high level languages that follow the **C** style syntax. **if** conditions, **for** and **while** loops, and **function** returns are similar in syntax to languages such as **C, C++, C#, Java** and **Perl**.

5. The hello world

```
<!DOCTYPE html>
<meta charset=utf-8>
<title>PHP Test</title>
<?php   echo 'Hello World';?>
```

6. With HTML elements

```
<!DOCTYPE html>
<meta charset=utf-8>
<title>PHP Test</title>
<?php echo "this is a  <strong>text</strong>"; ?>
```

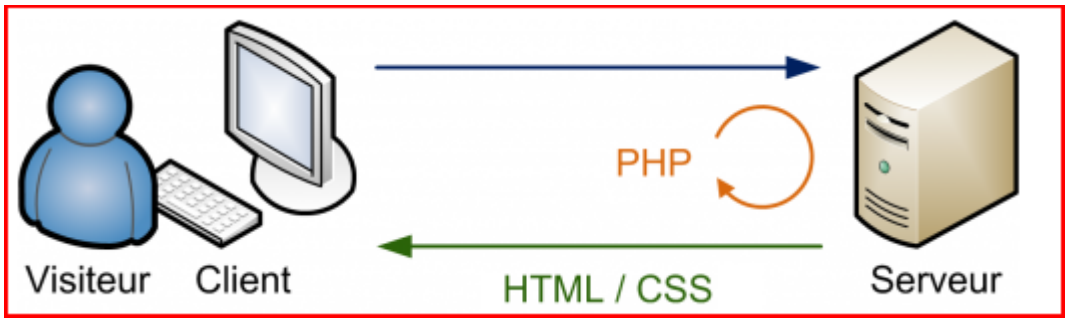
7. A Complete PHP page

```
<!DOCTYPE html >
  <head>
    <title>first instruction : echo</title>
    <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
  </head>
  <body>
    <h2>display with PHP</h2>
    <p>
      this line is entirely in HTML.<br />
      <?php echo " this line is entirely in PHP."; ?>
    </p>
  </body>
</html>
```

8. Try it and watch the source code

Name this file as test.html and try it with your web browser, and visualize the code sent by the by the server to the

Adress http://localhost/test/test.html



There is a page generation through the web server

PHP is making the page generation in background, before the server (host) to send the web page to the browser.

In short PHP interprets the hole page and translate the hole PHP-script inside before handing over the page to the server, which forwards it to the client machine, then its application browser can display the page, Finally the user (visitor)can watch it.

9. Simple example

Imagine an application requiring registration of some personal data from visitors and which should be returned as confirmation of accepted data. This simple task cannot be done by use of HTML only because the response must be adjusted to the submitted data. Figure 2.3 outlines the application in a diagram. The diagram indicates how the communications between the user and the host pass through the web server to the PHP scripts because the server cannot process the in data but is needed to return the web pages to the user for display. To summarize the task:

1. Design a HTML form for acquiring the required data
2. Develop a PHP script for returning a confirmation of received data

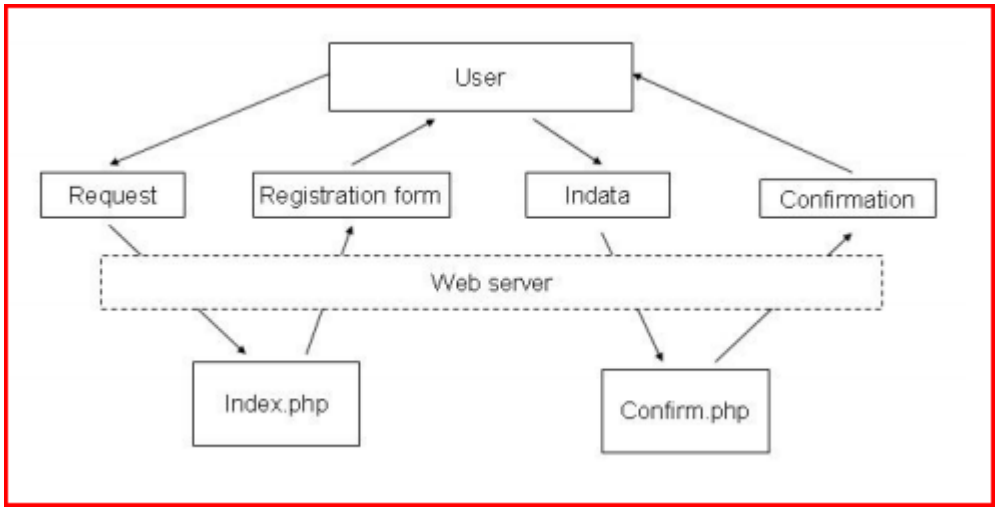


Figure 2.3 Diagram for registration

The development of a HTML form, may result in a typical file as:

```
<html>
<head>
<title>Registration</title>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
</head>
<body>
< !-- index.html -->
<center>
        <h2><font color="blue">Registration form</font></h2>
<form action="confirm.php" method="post">
<table>
<tr><td>First name:</td><td><input name="FirstName" type="text" size="30"></td></tr>
<tr><td>Last name:</td><td><input name="LastName" type="text" size="30"></td></tr>
<tr><td>Email adress:</td><td><input name="Email" type="text" size="28"></td></tr>
<tr><td></td><td><input name="" type="submit" value="Submit"></td></tr>
</table>
</form>
</center>
</body>
</html>
```

The page code specifies 3 text input fields and a submit button. There is nothing special with this code. Note that the statement in Line 10 has an ACTION attribute with value confirm.php implying that the control is transferred to a PHP file. This tells us that an application can consist of a mixture of .htm and .php files. By convention in this course, the first file of any example is either named index.html or index.php.

The purpose of the next file, confirm.php, is to instruct the server to return a confirmation for the received data. Except for a comment line including the name of the file, it contains a short PHP script. A PHP script is recognized by the start tag <?php and the end tag ?>. The script is short, but introduces several basic PHP Language characteristics.

```
1.  <!-- confirm.php -->
2.  <?php
3.  print("<center>");
4.  print ("<h3><font color=blue>The following data have been received:</font></h3>");
5.  print("<table>");
6.  print ("<tr><td>First name:</td><td> $_POST[FirstName]</td></tr>");
7.  print ("<tr><td>Last name: </td><td>$_POST[LastName]</td></tr>");
8.  print ("<tr><td>Email address:</td> <td>$_POST[Email]</td></tr>");
9.  print("</table>");
10.  print("</center>");
11.  ?>
```

These are:

Each PHP statement line ends with semicolon.

print() functions are used to return a message to the client.

$_POST[] elements are used to refer to values submitted in a form.

Let us first explain the $_POST[]. All variables in PHP are recognized by $ as their first character of their name. A name followed by [] indicates an array, and the content of the square parentheses refers to the key for the element the value of which is contained in the expression.

$_POST[] is an auto-global array in which all variable values submitted in a HTML FORM tag with METHOD="post". This means that the PHP automatically stores the variables in the array and that they are available to all parts of the application (Page).

Note that in PHP the elements of the returned HTML page must be enclosed as arguments surrounded by double quotes in the print functions. If you replace the $_POST[] variables with the values the contain, remove the print, parentheses, quotes and semicolons, you get the HTML page returned to the client by the web server. See the Source code in your browser when running the example.

VARIABLE

1. Description

Variables are nothing but identifiers to the memory location to store data. We can create any number of varibles. In PHP all the variables begin with a dollar sign "$" and the value can be assigned using the "=" operator as shown below:

Example:

```
$Name, $Name1, $First_Name, $Last_Name;
$Name = "David";
$Age = 16;
```

The main way to store information in the middle of a PHP program is by using a variable. Here are the most important things to know about variables in PHP.

- All variables in PHP are denoted with a leading dollar sign ($).
- The value of a variable is the value of its most recent assignment.
- Variables are assigned with the = operator, with the variable on the left-hand side and the expression to be evaluated on the right.
- Variables can, but do not need, to be declared before assignment.
- Variables in PHP do not have intrinsic types - a variable does not know in advance whether it will be used to store a number or a string of characters.
- Variables used before they are assigned have default values.
- PHP does a good job of automatically converting types from one to another when necessary.
- PHP variables are Perl-like.

2. Data types

PHP has a total of eight data types which we use to construct our variables:

2.1. Integers:

Are whole numbers, without a decimal point, like 4195.

Integers are whole numbers, without a decimal point, like 4195. They are the simplest type .they correspond to simple whole numbers, both positive and negative. Integers can be assigned to variables, or they can be used in expressions, like so:

```
$int_var = 12345;
$another_int = -12345 + 12345;
```

Integer can be in decimal (base 10), octal (base 8), and hexadecimal (base 16) format. Decimal format is the default, octal integers are specified with a leading 0, and hexadecimals have a leading 0x. For most common platforms, the largest integer is (2^31 . 1) (or 2,147,483,647), and the smallest (most negative) integer is . (2^31 . 1) (or .2,147,483,647).

2.2. Doubles

Are floating-point numbers, like 3.14159 or 49.1.

They like 3.14159 or 49.1. By default, doubles print with the minimum number of decimal places needed. For example, the code:

```
$many = 2.2888800;
$many_2 = 2.2111200;
$few = $many + $many_2;
print($many + $many_2 = $few<br>);
```

It produces the following browser output:

2.28888 + 2.21112 = 4.5

2.3. Booleans:

They have only two possible values either true or false.

They have only two possible values either true or false. PHP provides a couple of constants especially for use as Booleans: TRUE and FALSE, which can be used like so:

```
if (TRUE)
  print("This will always print<br>");
else
  print("This will never print<br>");
```

2.4. NULL

Is a special type that only has one value: NULL.

NULL is a special type that only has one value: NULL. To give a variable the NULL value, simply assign it like this:

```
$my_var = NULL;
```

The special constant NULL is capitalized by convention, but actually it is case insensitive; you could just as well have typed:

```
$my_var = null;
```

A variable that has been assigned NULL has the following properties:

- It evaluates to FALSE in a Boolean context.
- It returns FALSE when tested with IsSet() function.

2.5. Strings

Are sequences of characters, like 'PHP supports string operations.'

They are sequences of characters, like "PHP supports string operations". Following are valid examples of string

```
$string_1 = "This is a string in double quotes";
$string_2 = "This is a somewhat longer, singly quoted string";
$string_39 = "This string has thirty-nine characters";
$string_0 = ""; // a string with zero characters
```

Singly quoted strings are treated almost literally, whereas doubly quoted strings replace variables with their values as well as specially interpreting certain character sequences.

```
<?
$variable = "name";
$literally = 'My $variable will not print!\\n';
print($literally);
$literally = "My $variable will print!\n";
print($literally);
?>
```

This will produce following result:

My $variable will not print!\n

My name will print

There are no artificial limits on string length - within the bounds of available memory, you ought to be able to make arbitrarily long strings.

Strings that are delimited by double quotes (as in "this") are preprocessed in both the following two ways by PHP:

- Certain character sequences beginning with backslash (\) are replaced with special characters
- Variable names (starting with $) are replaced with string representations of their values.

The escape-sequence replacements are:

- \n is replaced by the newline character
- \r is replaced by the carriage-return character
- \t is replaced by the tab character
- \$ is replaced by the dollar sign itself ($)
- \" is replaced by a single double-quote (")
- \\ is replaced by a single backslash (\)

2.6. Arrays

Are named and indexed collections of other values.

2.7. Objects:

Objects are instances of programmer-defined classes, which can package up both other kinds of values and functions that are specific to the class.

2.8. Resources

Resources: are special variables that hold references to resources external to PHP (such as database connections).

The first five are simple types, and the next two (arrays and objects) are compound - the compound types can package up other arbitrary values of arbitrary type, whereas the simple types cannot.

3. Interpreting other types as Booleans apart from INTERGER and DOUBLE:

Here are the rules for determine the "truth" of any value not already of the Boolean type:

- If the value is a number, it is false if exactly equal to zero and true otherwise.
- If the value is a string, it is false if the string is empty (has zero characters) or is the string "0", and is true otherwise.
- Values of type NULL are always false.
- If the value is an array, it is false if it contains no other values, and it is true otherwise. For an object, containing a value means having a member variable that has been assigned a value.
- Valid resources are true (although some functions that return resources when they are successful will return FALSE when unsuccessful).
- Don't use double as Booleans.

Text:

```
<?php
$age_du_visiteur = 17;
echo ' visitor is $age_du_visiteur ans'; // Don’t work
?>

<?php
$age_du_visiteur = 17;
echo ' visitor is ' . $age_du_visiteur . ' ans';
?>
```

PHP File Inclusion

You can include the content of a PHP file into another PHP file before the server executes it. There are two PHP functions which can be used to included one PHP file into another PHP file.

- The include() Function
- The require() Function

This is a strong point of PHP which helps in creating functions, headers, footers, or elements that can be reused on multiple pages. This will help developers to make it easy to change the layout of complete website with minimal effort. If there is any change required then instead of changing thousand of files just change included file.

1. The include() Function

The include() function takes all the text in a specified file and copies it into the file that uses the include function. If there is any problem in loading a file then the **include()** function generates a warning but the script will continue execution.

Assume you want to create a common menu for your website. Then create a file menu.php with the following content.

1.1. Example1

The file form.php

```
<h2><font color="blue">Registration form</font></h2>
 <form action="confirm.php" method="get">
        <table>
         <tr><td>First name:</td><td><input name="FirstName" type="text" size="30"></td></tr>
         <tr><td>Last name:</td><td><input name="LastName" type="text" size="30"></td></tr>
         <tr><td>Email adress:</td><td><input name="Email" type="text" size="28"></td></tr>
         <tr><td></td><td><input name="" type="submit" value="Submit"></td></tr>
        </table>
 </form>
```

Is called this way in another HTML file.

```
<html>
        <body>
                <!-- What is computer science? -->
                        <a href="#my_anchor">write to author</a><!-- take me to anchor --
                        ><br/>
                        <a href="test1.html">Go back to the first form </a><!-- take me to
                        original -->
                <h1>the form!</h1>
                <p>
                        <hr />
                                <?php
                                        include "form1.php";
                                ?>
                        <hr />
                </p>
                <h2>Databases and information retrieval</h2>
...
```

1.2. Example2

```
<a href="http://www.tutorialspoint.com/index.htm">Home</a> -
<a href="http://www.tutorialspoint.com/ebxml">ebXML</a> -
<a href="http://www.tutorialspoint.com/ajax">AJAX</a> -
<a href="http://www.tutorialspoint.com/perl">PERL</a> <br />
```

Now create as many pages as you like and include this file to create header. For example now your test.php file can have following content.

```
<html>
<body>
<?php include("menu.php"); ?>
<p>This is an example to show how to include PHP file!</p>
</body>
</html>
```

This will produce following result

Home - ebXML - AJAX - PERL

This is an example to show how to include PHP file. You can include mean.php file in as many as files you like!

1.3. Error for include when a the file loops.php is missing

Warning: include(loops.php) [function.include]: failed to open stream: No such file or directory in **C:\xampp\htdocs\CEC409\pratical\test.php** on line **28**
Warning: include() [function.include]: Failed opening 'loops.php' for inclusion (include_path='.;C:\xampp\php\pear\')
in **C:\xampp\htdocs\CEC409\pratical\test.php** on line **28**

2. The require() Function

The **require()** function takes all the text in a specified file and copies it into the file that uses the include function. If there is any problem in loading a file then, the **require()** function generates a fatal error and halt the execution of the script.

So there is no difference in require() and include() except they handle error conditions. It is recommended to use the require() function instead of include(), because scripts should not continue executing if files are missing or misnamed.

You can try using above example with require() function and it will generate same result. But if you will try following two examples where file does not exist then you will get different results.

```
<html>
<body>
<?php include("xxmenu.php"); ?>
<p>This is an example to show how to include wrong PHP file!</p>
</body>
</html>
```

This will produce following result

```
This is an example to show how to include wrong PHP file!
```

Now lets try same example with require() function.

```
<html>
<body>
<?php require("xxmenu.php"); ?>
<p>This is an example to show how to include wrong PHP file!</p>
</body>
</html>
```

This time file execution halts and nothing is displayed.
NOTE: You may get plain warning messages or fatal error messages or nothing at all. This depends on your PHP Server configuration.

2.1. Example 2

```
<html>
<body>
<!-- What is computer science? -->
  <a href="#my_anchor">write to author</a><!-- take me to anchor --><br/>
        <a href="test1.html">Go back to the first form </a><!-- take me to original -->
        <h1>the form!</h1>

        <!-- include Form1-->
        <p>
        <hr />
        <?php
        include ("form1.php");
        echo "today is:".date("Y-m-d");
        ?>
        <hr />
        </p>
        <!-- include Mail1-->
        <p>
        <hr />
        <?php
        //include "mail.php";
        require ("loops.php");
        ?>
        <hr />
        </p>
        <h2>Databases and information retrieval</h2>
</body>
</html>
```

2.2. Error for require when a the file loops.php is missing

Warning: require(loops.php) [function.require]: failed to open stream: No such file or directory in **C:\xampp\htdocs\CEC409\pratical\test.php** on line **28**
Fatal error: require() [function.require]: Failed opening required 'loops.php' (include_path='.;C:\xampp\php\pear\')
in **C:\xampp\htdocs\CEC409\pratical\test.php** on line **28**

NB: You'll have nothing displayed after this fatal error line

3. Require VS Include

The require statement is also used to include a file into the PHP code.
However, there is one big difference between include and require; when a file is included with the include statement and PHP cannot find it, the script will continue to execute. If we do the same example using the require statement, the echo statement will not be

executed because the script execution dies after the require statement returned a fatal error.

4. PHP require_once(), include_once()

require_once() statement can be used to include a php file in another one, when you may need to include the called file more than once. If it is found that the file has already been included, calling script is going to ignore further inclusions.

```
require_once('name of the calling file with path');
```

The include_once() statement can be used to include a php file in another one, when you may need to include the called file more than once. If it is found that the file has already been included, calling script is going to ignore further inclusions.

```
include_once('name of the called file with path');
```

CONDITIONS

5. Decision Making

The if, elseif ...else and switch statements are used to take decision based on the different condition.

You can use conditional statements in your code to make your decisions. PHP supports following three decision making statements:

- **if...else statement** - use this statement if you want to execute a set of code when a condition is true and another if the condition is not true
- **elseif statement** - is used with the if...else statement to execute a set of code if **one** of several condition are true
- **switch statement** - is used if you want to select one of many blocks of code to be executed, use the Switch statement. The switch statement is used to avoid long blocks of if..elseif..else code.

6. The If...Else Statement

If you want to execute some code if a condition is true and another code if a condition is false, use the if....else statement.

6.1. Syntax

```
if (condition)
  code to be executed if condition is true;
else
  code to be executed if condition is false;
```

7. The ElseIf Statement

If you want to execute some code if one of several conditions are true use the elseif statement

7.1. Syntax

```
if (condition)
  code to be executed if condition is true;
elseif (condition)
  code to be executed if condition is true;
else
  code to be executed if condition is false;
```

7.2. Example

The following example will output "Have a nice weekend!" if the current day is Friday, and "Have a nice Sunday!" if the current day is Sunday. Otherwise it will output "Have a nice day!":

```
<html>
<body>
<?php
$d=date("D");
if ($d=="Fri")
  echo "Have a nice weekend!";
elseif ($d=="Sun")
  echo "Have a nice Sunday!";
else
  echo "Have a nice day!";
?>
</body>
</html>
```

8. The Switch Statement

If you want to select one of many blocks of code to be executed, use the Switch statement.

The switch statement is used to avoid long blocks of if..elseif..else code.

8.1. Syntax

```
switch (expression)
{
case label1:
  code to be executed if expression = label1;
  break;
case label2:
  code to be executed if expression = label2;
  break;
default:
  code to be executed
  if expression is different
  from both label1 and label2;

}
```

8.2. Example

The *switch* statement works in an unusual way. First it evaluates given expression then seeks a label to match the resulting value. If a matching value is found then the code

associated with the matching label will be executed or if none of the labels match, then statement will execute any specified default code.

```
<html>
<body>
<?php
$d=date("D");
switch ($d)
{
case "Mon":
 echo "Today is Monday";
 break;
case "Tue":
 echo "Today is Tuesday";
 break;
case "Wed":
 echo "Today is Wednesday";
 break;
case "Thu":
 echo "Today is Thursday";
 break;
case "Fri":
 echo "Today is Friday";
 break;
case "Sat":
 echo "Today is Saturday";
 break;
case "Sun":
 echo "Today is Sunday";
 break;
default:
 echo "Wonder which day is this ?";
}
?>
</body>
</html>
```

THE LOOP

Loops in PHP are used to execute the same block of code a specified number of times. PHP supports following four loop types.

- **for -** loops through a block of code a specified number of times.
- **while -** loops through a block of code if and as long as a specified condition is true.
- **do...while -** loops through a block of code once, and then repeats the loop as long as a special condition is true.

- **foreach -** loops through a block of code for each element in an array.

We will discuss about **continue** and **break** keywords used to control the loops execution.

9. The for loop statement

The for statement is used when you know how many times you want to execute a statement or a block of statements.

9.1. Syntax

```
for (initialization; condition; increment)
{
  code to be executed;
}
```

The initializer is used to set the start value for the counter of the number of loop iterations. A variable may be declared here for this purpose and it is traditional to name it $i.

9.2. Example

The following example makes five iterations and changes the assigned value of two variables on each pass of the loop:

```
<html>
<body>
<?php
$a = 0;
$b = 0;

for( $i=0; $i<5; $i++ )
{
   $a += 10;
   $b += 5;
}
echo ("At the end of the loop a=$a and b=$b" );
?>
</body>
</html>
```

This will produce following result:

```
At the end of the loop a=50 and b=25
```

10. The while loop statement

The while statement will execute a block of code if and as long as a test expression is true.

If the test expression is true then the code block will be executed. After the code has executed the test expression will again be evaluated and the loop will continue until the test expression is found to be false.

10.1. Syntax

```
while (condition)
{
   code to be executed;
}
```

10.2. Example

This example decrements a variable value on each iteration of the loop and the counter increments until it reaches 10 when the evaluation is false and the loop ends.

```
<html>
<body>
<?php
$i = 0;
$num = 50;

while( $i < 10)
{
  $num--;
  $i++;
}
echo ("Loop stopped at i = $i and num = $num" );
?>
</body>
</html>
```

This will produce following result:

```
Loop stopped at i = 10 and num = 40
```

11. The do...while loop statement

The **do...while** statement will execute a block of code at least once, it then will repeat the loop as long as a condition is true.

11.1. Syntax

```
Do
{
  code to be executed;
}while (condition);
```

11.2. Example

The following example will increment the value of i at least once, and it will continue incrementing the variable i as long as it has a value of less than 10:

```
<html>
<body>
<?php
$i = 0;
$num = 0;
do
{
 $i++;
}while( $i < 10 );
echo ("Loop stopped at i = $i" );
?>
</body>
</html>
```

This will produce following result:

```
Loop stopped at i = 10
```

12. The foreach loop statement

The foreach statement is used to loop through arrays. For each pass the value of the current array element is assigned to $value and the array pointer is moved by one and in the next pass next element will be processed

12.1. Syntax

```
foreach (array as value)
{
   code to be executed;

}
```

12.2. Example

Try out following example to list out the values of an array.

```
<html>
<body>
<?php
$array = array( 1, 2, 3, 4, 5);
foreach( $array as $value )
{
  echo "Value is $value <br />";
}
?>
</body>
</html>
```

This will produce following result:

```
Value is 1
Value is 2
Value is 3
Value is 4
Value is 5
```

13. The break statement

The PHP **break** keyword is used to terminate the execution of a loop prematurely.

The **break** statement is situated inside the statement block. If gives you full control and whenever you want to exit from the loop you can come out. After coming out of a loop immediate statement to the loop will be executed.

13.1. Example

In the following example condition test becomes true when the counter value reaches 3 and loop terminates.

```
<html>
<body>

<?php
$i = 0;

while( $i < 10)
{
```

```
  $i++;
  if( $i == 3 )break;
}
echo ("Loop stopped at i = $i" );
?>
</body>
</html>
```

This will produce following result:

```
Loop stopped at i = 3
```

14. The continue statement

The PHP **continue** keyword is used to halt the current iteration of a loop but it does not terminate the loop.

Just like the **break** statement the **continue** statement is situated inside the statement block containing the code that the loop executes, preceded by a conditional test. For the pass encountering **continue** statement, rest of the loop code is skipped and next pass starts.

14.1. Example

In the following example loop prints the value of array but for which condition becomes true it just skip the code and next value is printed.

```
<html>
<body>
<?php
$array = array( 1, 2, 3, 4, 5);
foreach( $array as $value )
{
  echo "Value BEFORE is $value <br />";
  if( $value == 3 )continue;
  echo "Value is $value <br />";
}
?>
</body>
</html>
```

This will produce following result

```
Value is 1
Value is 2
```

```
Value is 4
Value is 5
```

ARRAY (TABLES)

An array is a special variable, which can hold more than one value at a time.

If you have a list of items (a list of car names, for example), storing the cars in single variables could look like this:

In PHP, there are three types of arrays:

- Indexed arrays - Arrays with a numeric index
- Associative arrays - Arrays with named keys
- Multidimensional arrays - Arrays containing one or more arrays

Here's a clear and structured **course note on PHP arrays with examples** that you can use for teaching or self-study:

📘 PHP Arrays – Course Notes

1. Introduction

An **array** in PHP is a data structure that allows you to store multiple values in a single variable. Arrays are widely used for handling lists of data such as numbers, strings, or objects.

---

2. Types of Arrays in PHP

🔷 Indexed Arrays

- Store values with numeric keys (starting from 0).
- Example:

```
<?php
$fruits = array("Apple", "Banana", "Orange");
echo $fruits[0]; // Output: Apple
?>
```

🔷 Associative Arrays

- Store values with custom string keys.
- Example:

```
<?php
$student = array("name" => "John", "age" => 21, "grade" => "A");
echo $student["name"]; // Output: John
?>
```

🔷 Multidimensional Arrays

- Arrays inside arrays (like tables or matrices).
- Example:

```
<?php
$students = array(
    array("John", 21, "A"),
    array("Mary", 22, "B"),
    array("Peter", 20, "C")
);
echo $students[1][0]; // Output: Mary
?>
```

---

3. Array Functions

PHP provides many built-in functions to manipulate arrays:

Function	Description	Example
`count()`	Returns number of elements	`count($fruits)`
`array_push()`	Adds elements to end	`array_push($fruits, "Mango")`
`array_pop()`	Removes last element	`array_pop($fruits)`
`array_merge()`	Combines arrays	`array_merge($fruits, $vegetables)`
`in_array()`	Checks if value exists	`in_array("Apple", $fruits)`
`array_keys()`	Returns all keys	`array_keys($student)`

---

4. Looping Through Arrays

Using `foreach`

```
<?php
$colors = array("Red", "Green", "Blue");
foreach ($colors as $color) {
    echo $color . "<br>";
}
?>
```

Using `for` (indexed arrays)

```
<?php
for ($i = 0; $i < count($fruits); $i++) {
    echo $fruits[$i] . "<br>";
}
?>
```

---

5. Practical Examples

Example 1: Sum of Numbers

```
<?php
$numbers = array(10, 20, 30, 40);
$sum = 0;
foreach ($numbers as $num) {
    $sum += $num;
}
echo "Total Sum: " . $sum; // Output: 100
?>
```

Example 2: Student Grades

```
<?php
$grades = array("John" => "A", "Mary" => "B", "Peter" => "C");
foreach ($grades as $name => $grade) {
    echo $name . " scored " . $grade . "<br>";
}
?>
```

---

6. Key Takeaways

- Arrays are essential for handling collections of data.
- Use **indexed arrays** for ordered lists, **associative arrays** for key-value pairs, and **multidimensional arrays** for complex structures.
- Mastering array functions and loops makes data manipulation efficient.

👉 Would you like me to extend this into a **ready-to-use teaching notebook (with exercises and "Try it Yourself" sections)**, so your students can practice directly in class?

1. INDEXED/Numbered Array

```php
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array("Volvo", "BMW", "Toyota");
$arrlength = count($cars);

for($x = 0; $x < $arrlength; $x++) {
    echo $cars[$x];
    echo "<br>";
}
?>

</body>
</html>
```

```php
<?php
// On crée notre array $prenoms
$prenoms = array ('François', 'Michel', 'Nicole', 'Véronique', 'Benoît');

// Puis on fait une boucle pour tout afficher :
for ($numero = 0; $numero < 5; $numero++)
{
    echo $prenoms[$numero] . '<br />'; // affichera $prenoms[0], $prenoms[
}
?>
```

2. Associative Array.

Associative arrays are arrays that use named keys that you assign to them.
There are two ways to create an associative array:

```php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");
```

or:

```php
$age['Peter'] = "35";
$age['Ben'] = "37";
$age['Joe'] = "43";
```

Example1

```
<!DOCTYPE html>
<html>
<body>

<?php
$age = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach($age as $x => $x_value) {
    echo "Key=" . $x . ", Value=" . $x_value;
    echo "<br>";
}
?>

</body>
</html>
```

3. PHP - Multidimensional Arrays

A multidimensional array is an array containing one or more arrays.

In a multidimensional array, each element in the main array can also be an array. And each element in the sub-array can be an array, and so on.

Example

In this example we create a multidimensional array, with automatically assigned ID keys:

```
$families = array (
        "Griffin"=>array
        (
                "Peter",
                "Lois",
                "Megan"
        ),
        "Quagmire"=>array
        (
                "Glenn"
        ),
        "Brown"=>array
        (
                "Cleveland",
                "Loretta",
                "Junior"
        )
);
```

The array above would look like this if written to the output:

```
Array
(
[Griffin] => Array
 (
```

```
        [0] => Peter
        [1] => Lois
        [2] => Megan
 )
[Quagmire] => Array
 (
        [0] => Glenn
 )
[Brown] => Array
 (
        [0] => Cleveland
        [1] => Loretta
        [2] => Junior
 )
)
```

Lets try displaying a single value from the array above:

```
echo "Is " . $families['Griffin'][2] . " a part of the Griffin family?";
```

The code above will output:

```
Is Megan a part of the Griffin family?
```

Example2

```
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array
  (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
  );

for ($row = 0; $row < 4; $row++) {
  echo "<p><b>Row number $row</b></p>";
  echo "<ul>";
  for ($col = 0; $col < 3; $col++) {
    echo "<li>".$cars[$row][$col]."</li>";
  }
  echo "</ul>";
}
?>

</body>
</html>
```

Example3

```php
<!DOCTYPE html>
<html>
<body>

<?php
$cars = array
  (
  array("Volvo",22,18),
  array("BMW",15,13),
  array("Saab",5,2),
  array("Land Rover",17,15)
  );

echo $cars[0][0].": In stock: ".$cars[0][1].", sold: ".$cars[0][2].".<br>";
echo $cars[1][0].": In stock: ".$cars[1][1].", sold: ".$cars[1][2].".<br>";
echo $cars[2][0].": In stock: ".$cars[2][1].", sold: ".$cars[2][2].".<br>";
echo $cars[3][0].": In stock: ".$cars[3][1].", sold: ".$cars[3][2].".<br>";
?>

</body>
</html>
```

🔍 Searching in Arrays (PHP)

To **search within arrays in PHP**, you can use several built-in functions depending on whether you want to check for a value, find its position, or filter results. Here's a structured overview with examples:

1. `in_array()` – Check if a value exists

- Returns `true` if the value is found, otherwise `false`.

```php
<?php
$fruits = array("Apple", "Banana", "Orange");

if (in_array("Banana", $fruits)) {
    echo "Banana is in the array!";
} else {
    echo "Not found!";
}
?>
```

---

2. `array_search()` – Find the key of a value

- Returns the **key/index** of the first match, or `false` if not found.

```php
<?php
$fruits = array("Apple", "Banana", "Orange");

$key = array_search("Orange", $fruits);
if ($key !== false) {
    echo "Orange found at index: " . $key;
}
?>
```

---

3. `array_key_exists()` – Check if a key exists

- Useful for **associative arrays**.

```php
<?php
$student = array("name" => "John", "age" => 21, "grade" => "A");

if (array_key_exists("age", $student)) {
    echo "Age is set: " . $student["age"];
}
?>
```

---

4. Filtering with `array_filter()`

- Apply a condition to extract matching values.

```php
<?php
$numbers = array(10, 20, 30, 40, 50);

$result = array_filter($numbers, function($num) {
    return $num > 25; // keep numbers greater than 25
});

print_r($result);
// Output: Array ( [2] => 30 [3] => 40 [4] => 50 )
?>
```

---

5. Practical Example – Searching Student Grades

```php
<?php
$grades = array("John" => "A", "Mary" => "B", "Peter" => "C");

if (in_array("B", $grades)) {
    echo "At least one student scored B.<br>";
}

$key = array_search("A", $grades);
echo "Student with grade A: " . $key;
?>
```

✅ Key Takeaways

- Use `in_array()` for value existence.
- Use `array_search()` to get the key/index of a value.

- Use `array_key_exists()` to check if a key exists.
- Use `array_filter()` for conditional searches.

FUNCTIONS

PHP has hundreds of base functions and thousands more via extensions. These functions are well documented on the PHP site; however, the built-in library has a wide variety of naming conventions and inconsistencies. PHP currently has no functions for thread programming, although it does support multi process programming on POSIX systems.

Besides the built-in PHP functions, we can create our own functions.

A function is a block of statements that can be used repeatedly in a program. A function will not execute immediately when a page loads. A function will be executed by a call to the function.

Additional functions can be defined by a developer:

```
function myFunction() { // declares a function, this is named myFunction
 return 'John Doe'; // returns the value 'John Doe'
}

echo 'My name is ' . myFunction() . '!'; //outputs the text concatenated with the return value of myFunction.
// myFunction is called as a result of this syntax.
// The result of the output will be 'My name is John Doe!'
```

Example1

```
<!DOCTYPE html>
<html>
<body>

<?php
function familyName($fname, $year) {
    echo "$fname Refsnes. Born in $year <br>";
}

familyName("Hege","1975");
familyName("Stale","1978");
familyName("Kai Jim","1983");
?>

</body>
</html>
```

Example2

```php
<!DOCTYPE html>
<html>
<body>

<?php
function sum($x, $y) {
    $z = $x + $y;
    return $z;
}

echo "5 + 10 = " . sum(5,10) . "<br>";
echo "7 + 13 = " . sum(7,13) . "<br>";
echo "2 + 4 = " . sum(2,4);
?>

</body>
</html>
```

Major Functions in PHP Programming

Here's a concise **course note on key PHP functions** with small examples you can use in class or for self-study:

⚡ Key PHP Functions – Quick Guide

1. String Functions

`strlen()` – Length of a string

```php
<?php
echo strlen("Hello World"); // Output: 11
?>
```

`strtoupper()` – Convert to uppercase

```php
<?php
echo strtoupper("hello"); // Output: HELLO
?>
```

`strtolower()` – Convert to lowercase

```php
<?php
echo strtolower("HELLO"); // Output: hello
?>
```

---

2. Array Functions

`count()` – Number of elements

```php
<?php
$fruits = array("Apple", "Banana", "Orange");
```

```
echo count($fruits); // Output: 3
?>
```

`in_array()` – Check if value exists

```
<?php
if (in_array("Banana", $fruits)) {
    echo "Banana found!";
}
?>
```

`array_merge()` – Combine arrays

```
<?php
$a = array("Red", "Green");
$b = array("Blue", "Yellow");
print_r(array_merge($a, $b));
// Output: Array ( [0] => Red [1] => Green [2] => Blue [3] => Yellow )
?>
```

---

3. Math Functions

`max()` and `min()`

```
<?php
$numbers = array(10, 20, 5, 40);
echo max($numbers); // Output: 40
echo min($numbers); // Output: 5
?>
```

`round()`

```
<?php
echo round(3.14159, 2); // Output: 3.14
?>
```

---

4. Date & Time Functions

`date()`

```
<?php
echo date("Y-m-d H:i:s"); // Output: 2026-04-23 19:22:00 (example)
?>
```

`time()` – Current Unix timestamp

```
<?php
echo time(); // Output: e.g., 1766749320
?>
```

---

5. File Functions

`file_get_contents()` – Read file

```
<?php
$content = file_get_contents("example.txt");
echo $content;
?>
```

`file_put_contents()` – Write file

```
<?php
```

```
file_put_contents("example.txt", "Hello PHP!");
?>
```

---

6. Useful Utility Functions

`isset()` – Check if variable is set

```
<?php
$name = "John";
if (isset($name)) {
    echo "Variable exists!";
}
?>
```

`empty()` – Check if variable is empty

```
<?php
$var = "";
if (empty($var)) {
    echo "Variable is empty!";
}
?>
```

✅ Key Takeaways

- PHP has rich built-in functions for strings, arrays, math, dates, files, and variables.
- Mastering these makes coding faster and more efficient.
- Functions often return values that can be directly used in conditions or printed.

Quick Reference Cheatsheet

Most Used Functions:
1. `echo` / `print` - Output
2. `isset()` / `empty()` - Variable checking
3. `count()` - Array length
4. `strlen()` - String length
5. `date()` - Date formatting
6. `array_merge()` - Merge arrays
7. `file_get_contents()` - Read file
8. `json_encode()` / `json_decode()` - JSON handling
9. `include` / `require` - File inclusion
10. `session_start()` - Session handling

Security Critical:
- `htmlspecialchars()` - XSS protection
- `password_hash()` / `password_verify()` - Password security
- `filter_var()` - Input validation/sanitization
- `mysqli_real_escape_string()` / PDO prepared statements - SQL injection protection

Performance Critical:
- `array_key_exists()` vs `isset()` - Use appropriate one
- `strpos()` !== false vs `str_contains()` - String checking
- `foreach` vs array functions - Choose based on need

Debugging Essential:
- `var_dump()` - Detailed debugging
- `print_r()` - Readable array/object display
- `error_log()` - Log to error file
- `debug_backtrace()` - Call stack trace

This comprehensive list covers the major functions you'll use in PHP development. Remember to always check the official PHP documentation for the latest information and edge cases!

POST GET AND SESSION Methods

There are two ways the browser client can send information to the web server.

- The GET Method
- The POST Method

Before the browser sends the information, it encodes it using a scheme called URL encoding. In this scheme, name/value pairs are joined with equal signs and different pairs are separated by the ampersand.

```
name1=value1&name2=value2&name3=value3
```

Spaces are removed and replaced with the + character and any other non alphanumeric characters are replaced with a hexadecimal values. After the information is encoded it is sent to the server.

1. The GET Method

The GET method sends the encoded user information appended to the page request. The page and the encoded information are separated by the **?** character.

```
http://www.test.com/index.htm?name1=value1&name2=value2
```

- The GET method produces a long string that appears in your server logs, in the browser's Location: box.
- The GET method is restricted to send up to 1024 characters only.
- Never use GET method if you have password or other sensitive information to be sent to the server.
- GET can't be used to send binary data, like images or word documents, to the server.
- The data sent by GET method can be accessed using QUERY_STRING environment variable.
- The PHP provides **$_GET** associative array to access all the sent information using GET method.

Try out following example by putting the source code in test.php script.

```
<?php
 if( $_GET["name"] || $_GET["age"] )
 {
    echo "Welcome ". $_GET['name']. "<br />";
    echo "You are ". $_GET['age']. " years old.";
    exit();
 }
```

```
?>
<html>
<body>
 <form action="<?php $_PHP_SELF ?>" method="GET">
 Name: <input type="text" name="name" />
 Age: <input type="text" name="age" />
 <input type="submit" />
 </form>
</body>
</html>
```

2. The POST Method

The POST method transfers information via **HTTP** headers. The information is encoded as described in case of GET method and put into a header called QUERY_STRING.

- The POST method does not have any restriction on data size to be sent.
- The POST method can be used to send ASCII as well as binary data.
- The data sent by POST method goes through HTTP header so security depends on HTTP protocol. By using Secure HTTP you can make sure that your information is secured.
- The PHP provides **$_POST** associative array to access all the sent information using GET method.

Try out following example by putting the source code in test.php script.

```
<?php
 if( $_POST["name"] || $_POST["age"] )
 {
   echo "Welcome ". $_POST['name']. "<br />";
   echo "You are ". $_POST['age']. " years old.";
   exit();
 }
?>
<html>
<body>
 <form action="<?php $_PHP_SELF ?>" method="POST">

 Name: <input type="text" name="name" />
 Age: <input type="text" name="age" />

 <input type="submit" />
 </form>
</body>
</html>
```

3. The $_REQUEST variable

The PHP $_REQUEST variable contains the contents of both $_GET, $_POST, and $_COOKIE. We will discuss $_COOKIE variable when we will explain about cookies.

The PHP $_REQUEST variable can be used to get the result from **form** data sent with both the GET and POST methods.

Try out following example by putting the source code in test.php script.

```
<?php
 if( $_REQUEST["name"] || $_REQUEST["age"] )
 {
    echo "Welcome ". $_REQUEST['name']. "<br />";
    echo "You are ". $_REQUEST['age']. " years old.";
    exit();
 }
?>
<html>
<body>
 <form action="<?php $_PHP_SELF ?>" method="POST">

 Name: <input type="text" name="name" />
 Age: <input type="text" name="age" />

 <input type="submit" />
 </form>
</body>
</html>
```

Here **$_PHP_SELF** variable contains the name of self script in which it is being called.

4. SESSION

An alternative way to make data accessible across the various pages of an entire website is to use a PHP Session.

A session creates a file in a temporary directory on the server where registered session variables and their values are stored. This data will be available to all pages on the site during that visit.

The location of the temporary file is determined by a setting in the **php.ini** file called **session.save_path**. Before using any session variable make sure you have setup this path.

When a session is started following things happen:

- PHP first creates a unique identifier for that particular session which is a random string of 32 hexadecimal numbers such as 3c7foj34c3jj973hjkop2fc937e3443.
- A cookie called **PHPSESSID** is automatically sent to the user's computer to store unique session identification string.
- A file is automatically created on the server in the designated temporary directory and bears the name of the unique identifier prefixed by sess_ ie sess_3c7foj34c3jj973hjkop2fc937e3443.

When a PHP script wants to retrieve the value from a session variable, PHP automatically gets the unique session identifier string from the PHPSESSID cookie and then looks in its temporary directory for the file bearing that name and a validation can be done by comparing both values.

A session ends when the user loses the browser or after leaving the site, the server will terminate the session after a predetermined period of time, commonly 30 minutes duration.

📘 Sessions in PHP

1. What is a Session?

- A **session** is a way to store information (variables) across multiple pages.
- Unlike cookies (stored on the client's browser), session data is stored on the server.
- Each visitor gets a unique **session ID**.

---

2. Starting a Session

Use `session_start()` at the beginning of your PHP script before any output.

```php
<?php
session_start(); // Start the session
$_SESSION["username"] = "John"; // Store session variable
echo "Session started for " . $_SESSION["username"];
?>
```

---

3. Accessing Session Variables

You can access session variables on any page after starting the session.

```php
<?php
session_start();
echo "Welcome back, " . $_SESSION["username"];
?>
```

4. Modifying Session Variables

```php
<?php
session_start();
$_SESSION["username"] = "Mary"; // Change value
echo "Username changed to " . $_SESSION["username"];
?>
```

5. Removing Session Variables

- Use `unset()` to remove a specific variable.
- Use `session_destroy()` to end the session completely.

```php
<?php
session_start();
unset($_SESSION["username"]); // Remove one variable

session_destroy(); // End the session
echo "Session ended!";
?>
```

6. Practical Example – Login System

```php
<?php
// login.php
session_start();
if ($_POST["username"] == "admin" && $_POST["password"] == "1234") {
    $_SESSION["user"] = $_POST["username"];
    echo "Login successful!";
} else {
    echo "Invalid credentials!";
}
?>

<?php
// dashboard.php
session_start();
if (isset($_SESSION["user"])) {
    echo "Welcome, " . $_SESSION["user"];
} else {
    echo "Please login first!";
}
?>
```

✅ Key Takeaways

- Always call `session_start()` before using sessions.
- Sessions store data on the server, safer than cookies.
- Use `unset()` or `session_destroy()` to clear session data.
- Sessions are widely used for **login systems, shopping carts, and user preferences**.

🍪 Cookies in PHP

1. What is a Cookie?

- A **cookie** is a small piece of data stored on the client's browser.
- It helps remember information across multiple requests (e.g., user preferences, login status).
- Unlike sessions (stored on the server), cookies are stored on the client side.

---

2. Setting a Cookie

Use `setcookie()` before any output is sent to the browser.

```php
<?php
// setcookie(name, value, expire, path, domain, secure, httponly)
setcookie("username", "John", time() + (86400 * 7), "/"); // expires in 7 days
echo "Cookie set!";
?>
```

---

3. Accessing a Cookie

Cookies are available in the `$_COOKIE` superglobal array.

```php
<?php
if (isset($_COOKIE["username"])) {
    echo "Welcome back, " . $_COOKIE["username"];
} else {
    echo "No cookie found!";
}
?>
```

---

4. Modifying a Cookie

Just call `setcookie()` again with a new value.

```php
<?php
setcookie("username", "Mary", time() + (86400 * 7), "/");
echo "Cookie updated!";
?>
```

---

5. Deleting a Cookie

Set its expiration time to the past.

```php
<?php
setcookie("username", "", time() - 3600, "/");
echo "Cookie deleted!";
?>
```

---

6. Practical Example – Remembering User Preferences

```php
<?php
// preference.php
if (isset($_POST["theme"])) {
    setcookie("theme", $_POST["theme"], time() + (86400 * 30), "/");
    echo "Theme preference saved!";
}
?>

<?php
// homepage.php
if (isset($_COOKIE["theme"])) {
    echo "Your selected theme is: " . $_COOKIE["theme"];
} else {
    echo "Default theme applied.";
}
?>
```

✅ Key Takeaways

- Use `setcookie()` before any HTML output.
- Cookies store data on the **client side**, sessions on the **server side**.
- Useful for **remembering preferences, login states, and personalization**.
- Always consider **security** (e.g., `httponly`, `secure` flags for sensitive cookies).

4.1. Starting a PHP Session:

A PHP session is easily started by making a call to the **session_start()** function. This function first checks if a session is already started and if none is started then it starts one. It is recommended to put the call to **session_start()** at the beginning of the page.

Session variables are stored in associative array called **$_SESSION[]**. These variables can be accessed during lifetime of a session.

The following example starts a session then register a variable called **counter** that is incremented each time the page is visited during the session.

Make use of **isset()** function to check if session variable is already set or not.

Put this code in a test.php file and load this file many times to see the result:

```
<?php
  session_start();
  if( isset( $_SESSION['counter'] ) )
  {
    $_SESSION['counter'] += 1;
  }
  else
  {
    $_SESSION['counter'] = 1;
  }
  $msg = "You have visited this page ".  $_SESSION['counter'];
  $msg .= "in this session.";
?>
<html>
<head>
<title>Setting up a PHP session</title>
</head>
<body>
<?php  echo ( $msg ); ?>
</body>
</html>
```

4.2. Destroying a PHP Session:

A PHP session can be destroyed by **session_destroy()** function. This function does not need any argument and a single call can destroy all the session variables. If you want to destroy a single session variable then you can use **unset()** function to unset a session variable.

Here is the example to unset a single variable:

```
<?php
  unset($_SESSION['counter']);
?>
```

Here is the call which will destroy all the session variables:

```
<?php
  session_destroy();
?>
```

4.3. Turning on Auto Session:

You don't need to call start_session() function to start a session when a user visits your site if you can set **session.auto_start** variable to 1 in **php.ini** file.

4.4. Sessions without cookies:

There may be a case when a user does not allow to store cookies on their machine. So there is another method to send session ID to the browser.

Alternatively, you can use the constant SID which is defined if the session started. If the client did not send an appropriate session cookie, it has the form session_name=session_id. Otherwise, it expands to an empty string. Thus, you can embed it unconditionally into URLs.

The following example demonstrates how to register a variable, and how to link correctly to another page using SID.

```
<?php
  session_start();

  if (isset($_SESSION['counter'])) {
    $_SESSION['counter'] = 1;
  } else {
    $_SESSION['counter']++;
  }
?>
  $msg = "You have visited this page ".  $_SESSION['counter'];
  $msg .= "in this session.";
  echo ( $msg );
<p>
To continue  click following link <br />
<a  href="nextpage.php?<?php echo htmlspecialchars(SID); >">
</p>
```

The **htmlspecialchars()** may be used when printing the SID in order to prevent XSS related attacks.

DATABASE WITH PHP

1. Introduction

In fact, PHP offers several ways to connect to a MySQL database.

- The extension **mysql_** : these are functions that provide access to a MySQL database and thus communicate with MySQL. Their name always starts with mysql_ . However, these functions are old and it is recommended not to use today.

- The extension **mysqli_** : these are improved access to MySQL functions. They offer more features and are more up to date.

- The **PDO** extension: it is a comprehensive tool that provides access to any type of database. It can therefore be used to connect to MySQL as well as PostgreSQL or Oracle.

Aspect	MySQL (Original)	MySQLi	PDO
Status	**DEPRECATED** (PHP 5.5+)	**Active**	**Active**
Type	Procedural only	Procedural & OOP	OOP (with procedural style)
Database Support	MySQL only	MySQL only	**12+ databases**
Prepared Statements	❌ No	✅ Yes	✅ Yes
Named Parameters	❌ No	❌ No (only ?)	✅ Yes
Transactions	Basic	✅ Full support	✅ Full support
Performance	Good	Better	Good
Security	Poor (manual escaping)	Good	**Best**
Fetch Modes	Limited	Multiple	**Most flexible**

MySQL, Oracle, PostgreSQL, MariaDB Comparison

Database	Type	License	Primary Use
MySQL	Relational (RDBMS)	GPL (Open Source)	Web applications, LAMP stack
Oracle	Relational (RDBMS)	Proprietary	Enterprise, large-scale systems
PostgreSQL	Object-Relational (ORDBMS)	PostgreSQL License (Open Source)	Complex applications, geospatial, analytics
MariaDB	Relational (RDBMS)	GPL (Open Source)	MySQL replacement, web applications

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Data Model	Table-based, fixed schema	Document, Key-Value, Column, Graph
Schema	Rigid, predefined	Flexible, dynamic
Scaling	Vertical (scale-up)	Horizontal (scale-out)
ACID Compliance	✅ Full ACID	⚠️ Often BASE (eventual consistency)
JOINs	✅ Supported	❌ Limited or none
Transactions	✅ Multi-record, complex	⚠️ Single-record or limited
Query Language	SQL (standard)	Varies by database
Best For	Complex queries, relationships	Large-scale, unstructured data

2. Using mysql_connect

2.1. Simple example using mysql_connect:

```php
mysql_connect("localhost", "root", ""); // Connexion to MySQL
mysql_select_db("logging"); // Selection of DATABASE logging

 // We create variables
$message = htmlentities ($_POST['message']);
$pseudo = htmlentities ($_POST['pseudo']);

// We add an entry in the table with the mysql_query
mysql_query("INSERT INTO tchat_c VALUES('', '$pseudo', '$message',now())");
```

2.2. And close it after:

```
90
91  <?php
92  mysql_close(); //  MySQL Deconnexion
93  ?>
```

3. Open a Connection to the MySQL Server using mysqli_

Use the PHP mysqli_connect() function to open a new connection to the MySQL server.Before we can access data in a database, we must open a connection to the MySQL server.

In PHP, this is done with the mysqli_connect() function.

3.1. Syntax

```
mysqli_connect(host,username,password,dbname);
```

Host	**Optional. Either a host name or an IP address**
Username	Optional. The MySQL user name
Password	Optional. The password to log in with
Dbname	Optional. The default database to be used when performing queries

Note: There are more available parameters, but the ones listed above are the most important.

In the following example we store the connection in a variable ($con) for later use in the script:

```
<?php
// Create connection
$con=mysqli_connect("example.com","peter","abc123","my_db");

// Check connection
if (mysqli_connect_errno($con))
 {
 echo "Failed to connect to MySQL: " . mysqli_connect_error();
 }
?>
```

3.2. Close a Connection

The connection will be closed automatically when the script ends. To close the connection before, use the mysqli_close() function:

```
<?php
$con=mysqli_connect("example.com","peter","abc123","my_db");

// Check connection
if (mysqli_connect_errno($con))
 {
 echo "Failed to connect to MySQL: " . mysqli_connect_error();
 }

mysqli_close($con);
?>
```

4. Using the PDO

As every type, to work with the database in PHP, you must first connect to it.

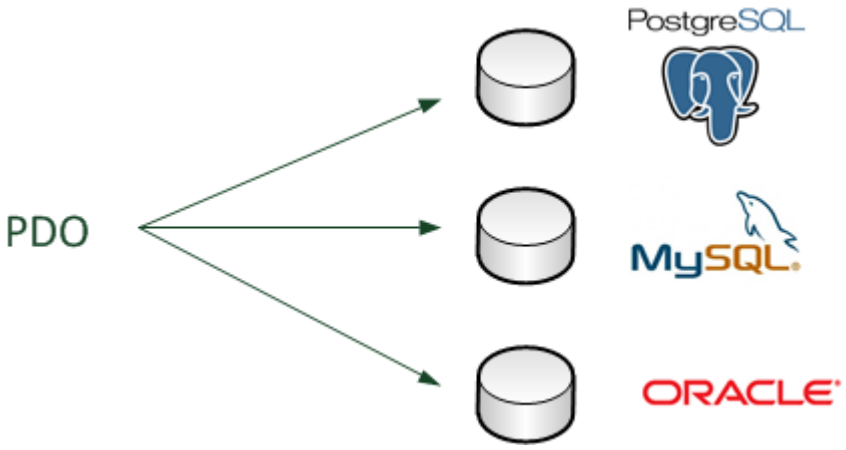
We will learn in this part to read data from a database (database). But I remind you that PHP should be the intermediary between you and MySQL. Problem: PHP can not tell MySQL early "Fetch me those values." Indeed, MySQL first request a user name and password. If he does not do it, everyone could access your DB and read information (sometimes confidential!) It contains.

So we have that authenticates PHP: it is said that *it establishes a connection* with MySQL. Once the connection is established, you can do all the things you want in your database!

These are all extensions as PHP is very modular. One can easily add or delete items to PHP, because everyone does not necessarily need all the features.

4.1. Which means choose among all these?

You'll understand the functions mysql_ are no longer used (they say they are "obsolete"). It remains to choose between mysqli_ and PDO. Here we will use PDO because it is the method of access to databases that will become the most used in future versions of PHP. On the other hand, the great advantage of PDO is that you can use the same way to connect to any other type of database (PostgreSQL, Oracle ...) (below).

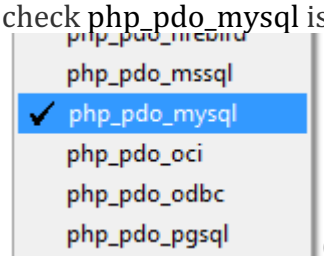


PDO can connect to any type of database

So you can reuse what you will learn if you choose to use another database MySQL.

4.2. Activate PDO

Normally, PDO is enabled by default. To verify (see figure below), left click on the WAMP icon in the taskbar, then go to the menu / PHP PHP Extensions and check php_pdo_mysql is checked.



Check that the PDO extension is enabled

NB: If not use WAMP
You can open the configuration file of PHP (usually called php.ini) and find the line that containsphp_pdo_mysql (line 3 in the example below). Remove the semicolon before if there is an extension to enable:

```
;extension=php_pdo_firebird.dll
;extension=php_pdo_mssql.dll
extension=php_pdo_mysql.dll
;extension=php_pdo_oci.dll
;extension=php_pdo_odbc.dll
```

If you're using Linux and you use XAMPP, locate the line that begins with pdo_mysql.default_socket and complete it as follows:
pdo_mysql.default_socket = / opt / lampp / var / mysql / mysql.sock
Save the file and restart PHP. Simply relaunch your favorite software (WAMP, MAMP, XAMPP ...).

4.3. Connect to MySQL with PDO

Now that we are certain that PDO is enabled, we can connect to MySQL. We will need four pieces of information:

- **the host name** : this is the address of the computer where MySQL is installed (such as an IP address). In most cases, MySQL is installed on the same computer as PHP: in this case, set the value localhost (meaning "on the same computer"). Nevertheless, it is possible that your web host tells you another value information (which look like this: sql.hebergeur.com). In this case, you should change this value when you submit your site to the Web;
- **Database** : this is the name of the database to which you want to connect. In our case, the database is named test . We have created with phpMyAdmin in the previous chapter;
- **login** : it allows you to identify. Check with your host to find out. In most cases (with a free provider) is the same login you use for FTP;
- **password** : there is a chance that the password is the same as the one you use to access the FTP. Check with your host.

For now, we test on our computer at home. It is said that work "locally". Therefore, the host name is localhost.
As the login and password, the default login is **root** and there is no password.
Here is how you should do to connect to MySQL via PDO based test:

```
<?php
$bdd = new PDO('mysql:host=localhost;dbname=test', 'root', '');
?>
```

We must recognize that it contains some new features. Indeed, PDO is called an extension **object-oriented** . It is a way to program a little different from traditional functions that we have learned to use so far.
We have the opportunity to learn about object-oriented programming (OOP) later in the book.For now, I invite you to reuse the code I suggest you follow my example. You understand the details of how they work later.

The line of code we just saw creates what is called an **object** $ db . This is not really a variable (although that's very similar): This is an object that represents the connection to the database. The connection is created by specifying in the order parameters:

- the host name(localhost);
- the database (test);
- the login (root) ;
- password (here there is no password, so I put an empty string).

When your site is online, you will surely have a different host name and a login and password as follows:

```
<?php
$bdd = new PDO('mysql:host=sql.hebergeur.com;dbname=mabase', 'pierre.durand',
's3cr3t');
?>
```

We must therefore consider changing this line to suit your host by modifying the information accordingly when submitting your site on the web.

The first parameter (starting with mysql) is called DSN: **D** ata **S** ource **N** ame. This is usually the only one that changes depending on the type of database to which it connects.

4.4. Test the presence of errors

If you have entered the correct information (hostname, database, login and password), nothing should be displayed on the screen. However, if there is an error (you have the wrong password or database name, for example), PHP may display the line that raises the error, including the password !

You do not want your visitors to see the password if an error occurs when your site is online. It is best to **treat** the error. In case of error, PDO returns a so-called **exception** that can "capture" the error.Here's how I suggest you do:

```
<?php
try
        {
           $bdd = new PDO('mysql:host=localhost;dbname=test', 'root', '');
        }
catch (Exception $e)
        {
             ( 'Erreur:' . $ e -> getMessage ());
        }
?>
```

Here is another code a bit new to us. Again, without going into too much detail, it should be noted that PHP tries to execute the instructions inside the block try. If an error occurs, it is within the block catch and did what we asked him (here the page execution is stopped with a message describing the error).

If instead all goes well, PHP continues to execute the code and do not read what's in the block catch .Your PHP page so nothing should show yet.

No, not at all! In fact, and I emphasize this, PDO us use of PHP that was not studied so far (OOP exceptions ...). Just keep to the time of reuse code that I offer and do not worry, we will return to these codes then later to explain in detail.

If you have a blank page, you can continue. If you get an error, check the message and try to understand what it means. If you get stuck, do not hesitate to ask for help on the forums , if you can not go further.

READ and WRITE DATA

1. INTRODUCTION

After the connection to database is successfully created and the PDO object instance is set, the object can be used to perform SQL queries. The SQL queries with PDO can be made in two ways:

- directly using "**exec()**", and "**query()**" methods,
- or with the **prepare() ... execute()** statement.

The first variant is more simple, in this lesson it's presented the **exec** method.

• The queries that modify rows in the table, but not return a result set with rows and columns (INSERT, UPDATE, and DELETE), are send with **exec()**, this method returns the number of affected rows, or FALSE on error.

```
$count = $conn->exec("SQL Query");
```

• Queries that select rows (SELECT) and return a result set with rows and columns are sent with the **query()** method. In case of error, returns FALSE.

```
$res = $conn->query("SQL Query");
```

To work with databases in PHP, you must know the specific SQL queries as: CREATE TABLE, INSERT, SELECT, UPDATE, etc.
These queries are send as a string to the MySQL server.

2. Create MySQL table

To create a table in a MySQL database, use the "***CREATE TABLE `table_name`***" query, and the ***exec()*** method:

$objPDO->exec("CREATE TABLE `table_name` (`column1` TYPE, `column2` TYPE, ...)");

All these instructions are added after the PDO object is created, containing the connection to MySQL database.

In the next example it is created in a database named "tests" a table named "sites", with 4 columnms: "id", "name", "category", and "link".

```
<?php
// Connection data (server_address, database, name, poassword)
$hostdb = 'localhost';
$namedb = 'tests';
$userdb = 'username';
$passdb = 'password';

try {
  // Connect and create the PDO object
  $conn = new PDO("mysql:host=$hostdb; dbname=$namedb", $userdb, $passdb);
```

```
 $conn->exec("SET CHARACTER SET utf8");      // Sets encoding UTF-8

 // Create the table
 $sql = "CREATE TABLE `sites` (
 `id` int(8) NOT NULL AUTO_INCREMENT PRIMARY KEY ,
 `name` varchar(70) NOT NULL DEFAULT '',
 `category` varchar(25),
 `link` varchar(100)
 ) CHARACTER SET utf8 COLLATE utf8_general_ci";
 if($conn->exec($sql) !== false) echo 'The sites table is created';
// If the result is not false, display confirmation

 $conn = null;       // Disconnect
}
catch(PDOException $e) {
  echo $e->getMessage();
}
?>
```

- If the table is created, the code above will display:

```
The sites table is created
```

- The instruction: "*exec("SET CHARACTER SET utf8")*" sets the transfer of data between PHP and MySQL to be made with UTF-8 encoding. It is advisable to add this instruction especialy when working with data containing diacritics or special characters, but the PHP script should also contain this header: **header('Content-type: text/html; charset=utf-8');**.

3. INSERT

Once the MySQL table is created, you can add rows with data. To add data into a table, use an INSERT command, in the exec() method.

Sintax:

$objPDO->exec("INSERT INTO `table_name` (`column1`, `column2`, ...) VALUES ('value1', 'value2', ...)");

Example:

```
<?php
// Connection data (server_address, database, name, poassword)
$hostdb = 'localhost';
$namedb = 'tests';
$userdb = 'username';
$passdb = 'password';

try {
 // Connect and create the PDO object
 $conn = new PDO("mysql:host=$hostdb; dbname=$namedb", $userdb, $passdb);
 $conn->exec("SET CHARACTER SET utf8");      // Sets encoding UTF-8

 // Define and insert query
 $sql = "INSERT INTO `sites` (`name`, `category`, `link`)
   VALUES
    ('Courses - Tutorials', 'education', 'coursesweb.net'),
```

```
    ('PHP-MySQL Course', 'programming', 'coursesweb.net/php-mysql'),
    ('English Courses', 'foreign languages', 'www.marplo.net/engleza')";
 $count = $conn->exec($sql);

 $conn = null;       // Disconnect
}
catch(PDOException $e) {
 echo $e->getMessage();
}

// If data added ($count not false) displays the number of rows added
if($count !== false) echo 'Number of rows added: '. $count;
?>
```

- This code adds 3 rows in the "sites" table. The $count variable stores the number of affected rows (added).

This script will display:

Number of rows added: 3

• To get the last auto-inserted "id" (in a AUTO_INCREMENT PRIMARY KEY column), use the lastInsertId() method.

$conn->lastInsertId();

- When you add multiple rows in the same INSERT query, this method will return the ID of the first added row.

UPDATE, and DELETE are SQL instructions that changes data in a table, but not return a result set with rows and columns. They can be executed in the same way as INSERT, with the exec() method.

4. UPDATE

The data in the rows of a MySQL table can be modified with the SQL command INSERT.

Syntax:

$objPDO->exec("UPDATE `table_name` SET `column1`='value1', `column2`='value2' WHERE condition");

The next example changes data in the columns "name" and "link", where "id" is 3; in the "sites" table (created with the code above).

```
<?php
// Connection data (server_address, database, name, poassword)
$hostdb = 'localhost';
$namedb = 'tests';
$userdb = 'username';
$passdb = 'password';

try {
 // Connect and create the PDO object
 $conn = new PDO("mysql:host=$hostdb; dbname=$namedb", $userdb, $passdb);
 $conn->exec("SET CHARACTER SET utf8");      // Sets encoding UTF-8
```

```
 // changes data in "name" si "link" columnns, where id=3
 $sql = "UPDATE `sites` SET `name`='Spanish Course', `link`='www.marplo.net/spaniola'
WHERE `id`=3";
 $count = $conn->exec($sql);

 $conn = null;       // Disconnect
}
catch(PDOException $e) {
 echo $e->getMessage();
}

// If the query is succesfully performed ($count not false)
if($count !== false) echo 'Affected rows : '. $count;      // Shows the number of affected rows
?>
```

- Result:

```
Affected rows : 1
```

Sometimes an UPDATE query not affect any row (if the condition not matches), and will return 0. So, it is indicated to use this statement to check the result: if($count !== false).

- Not: if(!$count)

5. DELETE

The DELETE instruction deletes rows in a table.

Syntax:

```
$objPDO->exec("DELETE FROM `table_name` WHERE condition");
```

The next example deletes all the rows in the "sites" table where the value in "category" column is "education" or "programming".

```
<?php
// Connection data (server_address, database, name, poassword)
$hostdb = 'localhost';
$namedb = 'tests';
$userdb = 'username';
$passdb = 'password';

try {
 // Connect and create the PDO object
 $conn = new PDO("mysql:host=$hostdb; dbname=$namedb", $userdb, $passdb);
 $conn->exec("SET CHARACTER SET utf8");     // Sets encoding UTF-8

 // Delete rows in "sites", according to the value of "category" column
 $sql = "DELETE FROM `sites` WHERE `category` IN('education', 'programming')";
 $count = $conn->exec($sql);

 $conn = null;       // Disconnect
}
catch(PDOException $e) {
 echo $e->getMessage();
}
```

```
// If the query is succesfully performed ($count not false)

if($count !== false) echo 'Affected rows: '. $count;      // Shows the number of aAffected rows

?>
```

- Result:

```
Affected rows: 2
```

6. PRACTICAL

Example1

```php
<?php
try
{
    $bdd = new PDO('mysql:host=localhost;dbname=test', 'root', '');
}
catch(Exception $e)
{
        die('Erreur : '.$e->getMessage());
}

$reponse = $bdd->query('SELECT * FROM person');

while ($donnees = $reponse->fetch())
{
    echo $donnees['name'] . '<br />';
}

$reponse->closeCursor();

?>
```

1. Inserting and displaying images in MySQL using PHP

Well working with images is quite easy task in MySQL using php code. Some years back managing images in relational database is quite complex task as at those times relational databases are able to store only textual data so file path to the images are stored in database and images are stored and retrieved externally. Also special file functions are necessary for retrieving images this way and this approach is system dependent (because of path names used). Nowadays, almost all major DBMS support storing of images directly in database by storing images in the form of binary data. Here, I am explaining the method of storing and retrieving images in MySQL database using PHP code.

2. Inserting images in mysql-:

MySQL has a blob data type which can be used to store binary data. A **blob** is a collection of binary data stored as a single entity in a database management system. Blobs are typically images, audio or other multimedia blob objects. MySQL has four BLOB types:

- TINYBLOB
- BLOB
- MEDIUMBLOB
- LONGBLOB

All these types differ only in their sizes.
For my demonstration, lets us create a test table named **test_image** in MySQL having 3 columns show below-:

- Id (INT) -Act as primary key for table.
- Name (VARCHAR) – Used to store image name.
- Image (BLOB) – Used to store actual image data.

You can use phpMyAdmin tool to create the above table else use the following MySQL query-:

```
create table test_image (
        id          int(10)  not null AUTO_INCREMENT PRIMARY KEY,
        name        varchar(25) not null default '',
        image       blob      not null
);
```

PHP code to upload image and store in database-:
To upload the image file from client to server and then store image in MySQL database on server, I am posting here the PHP code for our test/sample table (test_image).
Please change the values of variables in file_constants.php file according to your system. Save the following scripts with names as shown in your web directory.

3. file_constants.php

```
<html>
<head><title>File Insert</title></head>
```

```
<body>

<h3>Please Choose a File and click Submit</h3>

<form enctype="multipart/form-data" action=
"<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
<input type="hidden" name="MAX_FILE_SIZE" value="10000000" />
<input name="userfile" type="file" />
<input type="submit" value="Submit" />
</form>

<?php

// check if a file was submitted
if(!isset($_FILES['userfile']))
{
   echo '<p>Please select a file</p>';
}
else
{
   try {
   $msg= upload();  //this will upload your image
   echo $msg;  //Message showing success or failure.
   }
   catch(Exception $e) {
   echo $e->getMessage();
   echo 'Sorry, could not upload file';
   }
}

// the upload function

function upload() {
   include "file_constants.php";
   $maxsize = 10000000; //set to approx 10 MB

   //check associated error code
   if($_FILES['userfile']['error']==UPLOAD_ERR_OK) {

      //check whether file is uploaded with HTTP POST
      if(is_uploaded_file($_FILES['userfile']['tmp_name'])) {

         //checks size of uploaded image on server side
         if( $_FILES['userfile']['size'] < $maxsize) {

               // prepare the image for insertion
               $imgData =addslashes (file_get_contents($_FILES['userfile']['tmp_name']));
```

```
            // put the image in the db...
            // database connection
            mysql_connect($host, $user, $pass) OR DIE (mysql_error());

            // select the db
            mysql_select_db ($db) OR DIE ("Unable to select db".mysql_error());

            // our sql query
            $sql = "INSERT INTO test_image
            (image, name)
            VALUES
            ('{$imgData}', '{$_FILES['userfile']['name']}');";

            // insert the image
            mysql_query($sql) or die("Error in Query: " . mysql_error());
            $msg='<p>Image successfully saved in database with id ='. mysql_insert_id().' </p>';

                                        mysql_close();

         }
         else {
           // if the file is not less than the maximum allowed, print an error
           $msg='<div>File exceeds the Maximum File limit</div>
           <div>Maximum File limit is '.$maxsize.' bytes</div>
           <div>File '.$_FILES['userfile']['name'].' is '.$_FILES['userfile']['size'].
           ' bytes</div><hr />';
           }
     }
     else
        $msg="File not uploaded successfully.";

  }
  else {
     $msg= file_upload_error_message($_FILES['userfile']['error']);
  }
  return $msg;
}

// Function to return error message based on error code

function file_upload_error_message($error_code) {
  switch ($error_code) {
     case UPLOAD_ERR_INI_SIZE:
        return 'The uploaded file exceeds the upload_max_filesize directive in php.ini';
     case UPLOAD_ERR_FORM_SIZE:
```

```
        return 'The uploaded file exceeds the MAX_FILE_SIZE directive that was specified in the HTML form';
      case UPLOAD_ERR_PARTIAL:
        return 'The uploaded file was only partially uploaded';
      case UPLOAD_ERR_NO_FILE:
        return 'No file was uploaded';
      case UPLOAD_ERR_NO_TMP_DIR:
        return 'Missing a temporary folder';
      case UPLOAD_ERR_CANT_WRITE:
        return 'Failed to write file to disk';
      case UPLOAD_ERR_EXTENSION:
        return 'File upload stopped by extension';
      default:
        return 'Unknown upload error';
    }
}

?>
</body>
</html>
```

4. Read Image

```
<?PHP
include "file_constants.php";
// put the image in the db...
// database connection
mysql_connect($host, $user, $pass) OR DIE (mysql_error());

// select the db
mysql_select_db ($db) OR DIE ("Unable to select db".mysql_error());

// Read the row we want to pull out of the database.
$result = mysql_query("select * from test_image where id = 2");
$img = null;
// If successful, fetch the row as an array and store the data from the "image" column into a variable.
if ($result) {

   if ($row = mysql_fetch_array($result)) {
      $img = $row["image"];
   }
}
mysql_close();
```

```
// Set the content type of this page to image/jpeg since the image we are pulling out is a jpg image.

header("Content-type: image/jpeg");

// Echo out the image.
echo "$img";

?>
```

Ref :
http://www.coderslexicon.com/inserting-images-into-mysql-and-retrieving-them-using-php/

Object-Oriented PHP

5. Introduction

Object-oriented programming is a style of coding that allows developers to group similar tasks into classes. This helps keep code following the tenet "don't repeat yourself" (DRY) and easy-to-maintain.

"Object-oriented programming is a style of coding that allows developers to group similar tasks into classes."One of the major benefits of DRY programming is that, if a piece of information changes in your program, usually only one change is required to update the code.

OOP is intimidating to a lot of developers because it introduces new syntax and, at a glance, appears to be far more complex than simple procedural, or inline, code. However, upon closer inspection, OOP is actually a very straightforward and ultimately simpler approach to programming.

6. Understanding Objects and Classes

Before you can get too deep into the finer points of OOP, a basic understanding of the differences between objects and classes is necessary. This section will go over the building blocks of classes, their different capabilities, and some of their uses.

Recognizing the Differences Between Objects and Classes

Right off the bat, there's confusion in OOP: seasoned developers start talking about objects and classes, and they appear to be interchangeable terms. This is not the case, however, though the difference can be tough to wrap your head around at first.

A class, for example, is like a blueprint for a house. It defines the shape of the house on paper, with relationships between the different parts of the house clearly defined and planned out, even though the house doesn't exist.

An object, then, is like the actual house built according to that blueprint. The data stored in the object is like the wood, wires, and concrete that composes the house: without being assembled according to the blueprint, it's just a pile of stuff. However, when it all comes together, it becomes an organized, useful house.

7. Structuring Classes

The syntax to create a class is pretty straightforward: declare a class using the class keyword, followed by the name of the class and a set of curly braces ({}):

```
1.  <?php
2.
3.  class MyClass
4.  {
5.      // Class properties and methods go here
6.  }
7.
8.  ?>
```

After creating the class, a new class can be instantiated and stored in a variable using the new keyword:

```
1.  $obj = new MyClass;
```

To see the contents of the class, use var_dump():

```
1.  var_dump($obj);
```

Try out this process by putting all the preceding code in a new file called test.php in [your local] testing folder:

```
1.  <?php
2.
3.  class MyClass
4.  {
5.      // Class properties and methods go here
6.  }
7.
8.  $obj = new MyClass;
9.
10. var_dump($obj);
11.
12. ?>
```

8. Defining Class Properties

To add data to a class, **properties**, or class-specific variables, are used. These work exactly like regular variables, except they're bound to the object and therefore can only be accessed using the object.

To add a property to MyClass, add the following code to your script:

```
1.  <?php
2.
3.  class MyClass
4.  {
5.      public $prop1 = "I'm a class property!";
6.  }
7.
8.  $obj = new MyClass;
9.
10. var_dump($obj);
11.
12. ?>
```

The keyword **public** determines the visibility of the property, which you'll learn about a little later in this chapter. Next, the property is named using standard variable syntax, and a value is assigned (though class properties do not need an initial value).

To read this property and output it to the browser, reference the object from which to read and the property to be read:

```
1.  echo $obj->prop1;
```

Because multiple instances of a class can exist, if the individual object is not referenced, the script would be unable to determine which object to read from. The use of the arrow (->) is an OOP construct that accesses the contained properties and methods of a given object.
Modify the script in test.php to read out the property rather than dumping the whole class by modifying the code as shown:

```
<?php

class MyClass
{
   public $prop1 = "I'm a class property!";
}

$obj = new MyClass;

echo $obj->prop1; // Output the property

?>
```

Reloading your browser now outputs the following:

```
I'm a class property!
```

9. Defining Class Methods

Methods are class-specific functions. Individual actions that an object will be able to perform are defined within the class as methods.
For instance, to create methods that would set and get the value of the class property $prop1, add the following to your code:

```
<?php

class MyClass
{
   public $prop1 = "I'm a class property!";

   public function setProperty($newval)
   {
      $this->prop1 = $newval;
   }

   public function getProperty()
   {
      return $this->prop1 . "<br />";
   }
}

$obj = new MyClass;

echo $obj->prop1;

?>
```

Note — OOP allows objects to reference themselves using $this. When working within a method, use $this in the same way you would use the object name outside the class.

To use these methods, call them just like regular functions, but first, reference the object they belong to. Read the property from MyClass, change its value, and read it out again by making the modifications below:

```php
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

$obj = new MyClass;

echo $obj->getProperty(); // Get the property value

$obj->setProperty("I'm a new property value!"); // Set a new one

echo $obj->getProperty(); // Read it out again to show the change

?>
```

Reload your browser, and you'll see the following:

```
I'm a class property!
I'm a new property value!
```

"The power of OOP becomes apparent when using multiple instances of the same class."

```php
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

// Create two objects
$obj = new MyClass;
$obj2 = new MyClass;
```

```
21.
22. // Get the value of $prop1 from both objects
23. echo $obj->getProperty();
24. echo $obj2->getProperty();
25.
26. // Set new values for both objects
27. $obj->setProperty("I'm a new property value!");
28. $obj2->setProperty("I belong to the second instance!");
29.
30. // Output both objects' $prop1 value
31. echo $obj->getProperty();
32. echo $obj2->getProperty();
33.
34. ?>
```

When you load the results in your browser, they read as follows:

```
1. I'm a class property!
2. I'm a class property!
3. I'm a new property value!
4. I belong to the second instance!
```

10. Magic Methods in OOP

To make the use of objects easier, PHP also provides a number of **magic methods**, or special methods that are called when certain common actions occur within objects. This allows developers to perform a number of useful tasks with relative ease.

11. Using Constructors and Destructors

When an object is instantiated, it's often desirable to set a few things right off the bat. To handle this, PHP provides the magic method **__construct(),** which is called automatically whenever a new object is created.

For the purpose of illustrating the concept of constructors, add a constructor to MyClass that will output a message whenever a new instance of the class is created:

```
1.  <?php
2.
3.  class MyClass
4.  {
5.      public $prop1 = "I'm a class property!";
6.
7.      public function __construct()
8.      {
9.          echo 'The class "', __CLASS__, '" was initiated!<br />';
10.     }
11.
12.     public function setProperty($newval)
13.     {
14.         $this->prop1 = $newval;
15.     }
16.
17.     public function getProperty()
18.     {
19.         return $this->prop1 . "<br />";
```

```
    }
}

// Create a new object
$obj = new MyClass;

// Get the value of $prop1
echo $obj->getProperty();

// Output a message at the end of the file
echo "End of file.<br />";

?>
```

Note — __CLASS__ returns the name of the class in which it is called; this is what is known as a magic constant. There are several available magic constants, which you can read more about in the PHP manual.

Reloading the file in your browser will produce the following result:

```
The class "MyClass" was initiated!
I'm a class property!
End of file.
```

```
The class "MyClass" was initiated!
I'm a class property!
End of file.
```

To call a function when the object is destroyed, the **__destruct()** magic method is available. This is useful for class cleanup (closing a database connection, for instance).

Output a message when the object is destroyed by defining the magic method __destruct() in MyClass:

```
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function __construct()
    {
        echo 'The class "', __CLASS__, '" was initiated!<br />';
    }

    public function __destruct()
    {
        echo 'The class "', __CLASS__, '" was destroyed.<br />';
    }

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

// Create a new object
$obj = new MyClass;
```

```
30.
31. // Get the value of $prop1
32. echo $obj->getProperty();
33.
34. // Output a message at the end of the file
35. echo "End of file.<br />";
36.
37. ?>
```

With a destructor defined, reloading the test file results in the following output:

```
1. The class "MyClass" was initiated!
2. I'm a class property!
3. End of file.
4. The class "MyClass" was destroyed.
```

"When the end of a file is reached, PHP automatically releases all resources."

To explicitly trigger the destructor, you can destroy the object using the function unset():

```
1. <?php
2.
3. class MyClass
4. {
5.     public $prop1 = "I'm a class property!";
6.
7.     public function __construct()
8.     {
9.         echo 'The class "', __CLASS__, '" was initiated!<br />';
10.    }
11.
12.    public function __destruct()
13.    {
14.        echo 'The class "', __CLASS__, '" was destroyed.<br />';
15.    }
16.
17.    public function setProperty($newval)
18.    {
19.        $this->prop1 = $newval;
20.    }
21.
22.    public function getProperty()
23.    {
24.        return $this->prop1 . "<br />";
25.    }
26. }
27.
28. // Create a new object
29. $obj = new MyClass;
30.
31. // Get the value of $prop1
32. echo $obj->getProperty();
33.
34. // Destroy the object
35. unset($obj);
36.
37. // Output a message at the end of the file
38. echo "End of file.<br />";
39.
40. ?>
```

Now the result changes to the following when loaded in your browser:

```
The class "MyClass" was initiated!
I'm a class property!
The class "MyClass" was destroyed.
End of file.
```

12. Converting to a String

To avoid an error if a script attempts to output MyClass as a string, another magic method is used called__toString().

Without __toString(), *attempting to output the object as a string results in a fatal error*. Attempt to use echo to output the object without a magic method in place:

```php
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function __construct()
    {
        echo 'The class "', __CLASS__, '" was initiated!<br />';
    }

    public function __destruct()
    {
        echo 'The class "', __CLASS__, '" was destroyed.<br />';
    }

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

// Create a new object
$obj = new MyClass;

// Output the object as a string
echo $obj;

// Destroy the object
unset($obj);

// Output a message at the end of the file
echo "End of file.<br />";

?>
```

13. To avoid some errors, add a __toString() method:

```php
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function __construct()
    {
        echo 'The class "', __CLASS__, '" was initiated!<br />';
    }

    public function __destruct()
    {
        echo 'The class "', __CLASS__, '" was destroyed.<br />';
    }

    public function __toString()
    {
        echo "Using the toString method: ";
        return $this->getProperty();
    }

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

// Create a new object
$obj = new MyClass;

// Output the object as a string
echo $obj;

// Destroy the object
unset($obj);

// Output a message at the end of the file
echo "End of file.<br />";

?>
```

In this case, attempting to convert the object to a string results in a call to the getProperty() method. Load the test script in your browser to see the result:

```
The class "MyClass" was initiated!
Using the toString method: I'm a class property!
The class "MyClass" was destroyed.
End of file.
```

Tip — In addition to the magic methods discussed in this section, several others are available. For a complete list of magic methods, see the PHP manual page.

14. Using Class Inheritance

Classes can inherit the methods and properties of another class using the extends keyword. For instance, to create a second class that extends MyClass and adds a method, you would add the following to your test file:

```php
<?php

class MyClass
{
    public $prop1 = "I'm a class property!";

    public function __construct()
    {
        echo 'The class "', __CLASS__, '" was initiated!<br />';
    }

    public function __destruct()
    {
        echo 'The class "', __CLASS__, '" was destroyed.<br />';
    }

    public function __toString()
    {
        echo "Using the toString method: ";
        return $this->getProperty();
    }

    public function setProperty($newval)
    {
        $this->prop1 = $newval;
    }

    public function getProperty()
    {
        return $this->prop1 . "<br />";
    }
}

class MyOtherClass extends MyClass
{
    public function newMethod()
    {
        echo "From a new method in " . __CLASS__ . ".<br />";
    }
}

// Create a new object
$newobj = new MyOtherClass;

// Output the object as a string
echo $newobj->newMethod();

// Use a method from the parent class
echo $newobj->getProperty();

?>
```

15. EXAMPLE 2

```
class Person {
 public $firstName;
 public $lastName;

 public function __construct($firstName, $lastName = '') {
            //Optional parameter
  $this->firstName = $firstName;
  $this->lastName = $lastName;
 }

 public function greet() {
  return "Hello, my name is " . $this->firstName . " " . $this->lastName . ".";
 }

 public static function staticGreet($firstName, $lastName) {
  return "Hello, my name is " . $firstName . " " . $lastName . ".";
 }
}

$he = new Person('John', 'Smith');
$she = new Person('Sally', 'Davis');
$other = new Person('iAmine');

echo $he->greet(); // prints "Hello, my name is John Smith."
echo '<br />';
echo $she->greet(); // prints "Hello, my name is Sally Davis."
echo '<br />';
echo $other->greet(); // prints "Hello, my name is iAmine."
echo '<br />';
echo Person::staticGreet('Jane', 'Doe'); // prints "Hello, my name is Jane Doe."
```

The visibility of PHP properties and methods is defined using the keywords public, private, and protected. The default is public, if only var is used; var is a synonym for public.

Items declared **public** can be accessed everywhere.

Protected limits access to inherited classes (and to the class that defines the item).

Private limits visibility only to the class that defines the item.

Objects of the same type have access to each other's private and **protected** members even though they are not the same instance.

PHP's member visibility features have sometimes been described as "highly useful. However, they have also sometimes been described as "at best irrelevant and at worst positively harmful.

ORGANIZE THEIR CODE ACCORDING TO THE MVC ARCHITECTURE

The objective of this chapter is to introduce you to the MVC architecture, good programming practice that will help you properly design your future website. After reading, you will feel far more able to create a quality website and easy to maintain. ;

What is MVC?

You probably have many questions about the proper way to design your website. Let me reassure you about it: these questions, we're all of us asked one day. In fact, there are problems in programming returning so often we have created a series of best practices that have been gathered under the name of design patterns .

One of the most famous design patterns called MVC means **Model - View - Controller** . This is what we will discover in this chapter.

The MVC pattern can well organize your source code. It will help you know which files to create, but also to define their role. The purpose of MVC is precisely to separate the code logic into three parts found in separate files, as explained in the following description.

Model

This section manages the data of your website. Its role is to go retrieve information "raw" in the database, organize and assemble them so that they can then be processed by the controller. There are so SQL queries.

Sometimes the data are not stored in a database. That's rare, but it may be necessary to fetch data in files. In this case, the role of the model is to make the opening, reading and writing files operations (functions fopen , fgets , etc..).

View

This section focuses on the display . It does almost no calculation and simply retrieve variables to know what it should display. There are essentially HTML but also some very simple PHP loops and conditions to see for example the list of forum postings.

Controller

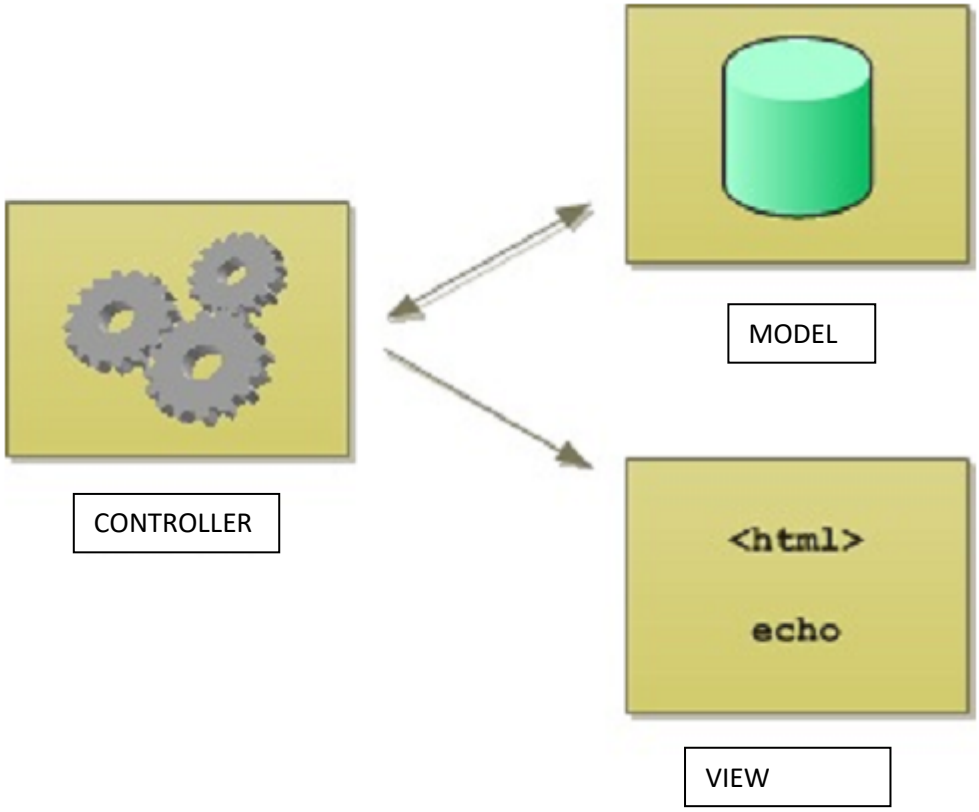
This section handles the logic of the code that makes decisions . This is somewhat intermediate between the model and the view: the controller will request the data model, analyze, make decisions and send the text to display in the view. The controller contains only PHP. This is particularly the one who determines if the visitor has the right to view the page or not (management of access rights).

The following figure shows the role of each of these elements.



The MVC architecture

It is important to understand how these elements fit together and communicate with each other. Look carefully in the following figure.



Exchange of information between elements

It must first be remembered that the controller is the conductor: it is the one who receives the request and visitor contacts other files (model and view) to exchange information with them.

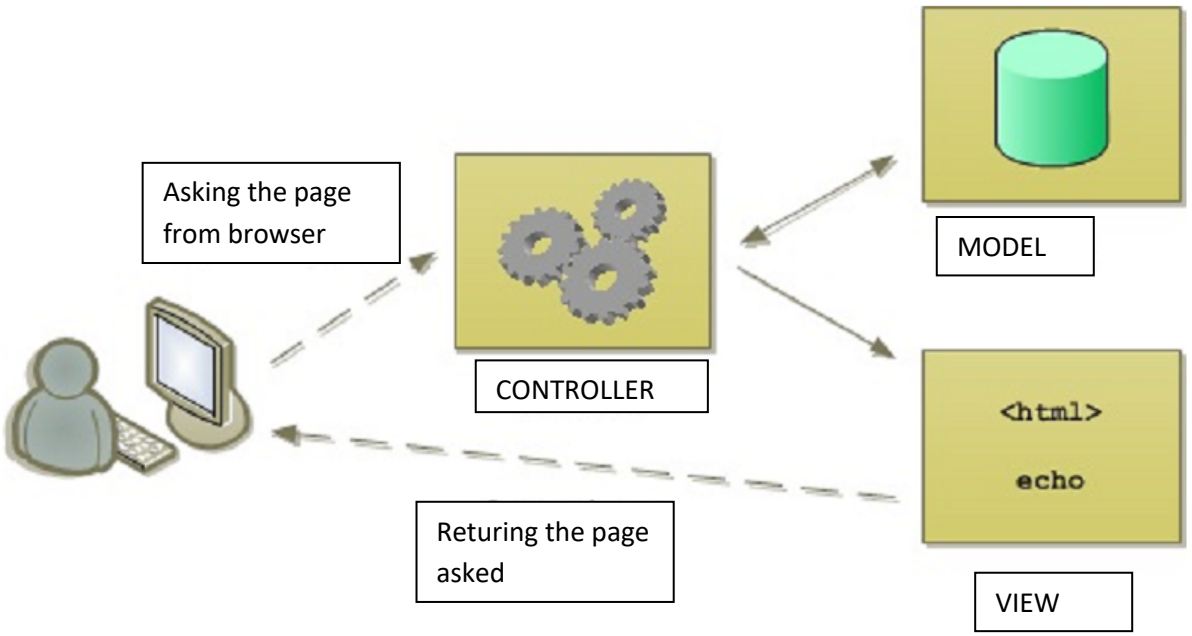
The file controller ask the model without worrying about how it will recover. For example: "Give me the list of the last 30messages posted in forum N 5." The model translates the request into a SQL query, retrieves the information and returns it to the controller.

Once the data is retrieved, the controller transmits to view who will display a list of messages.

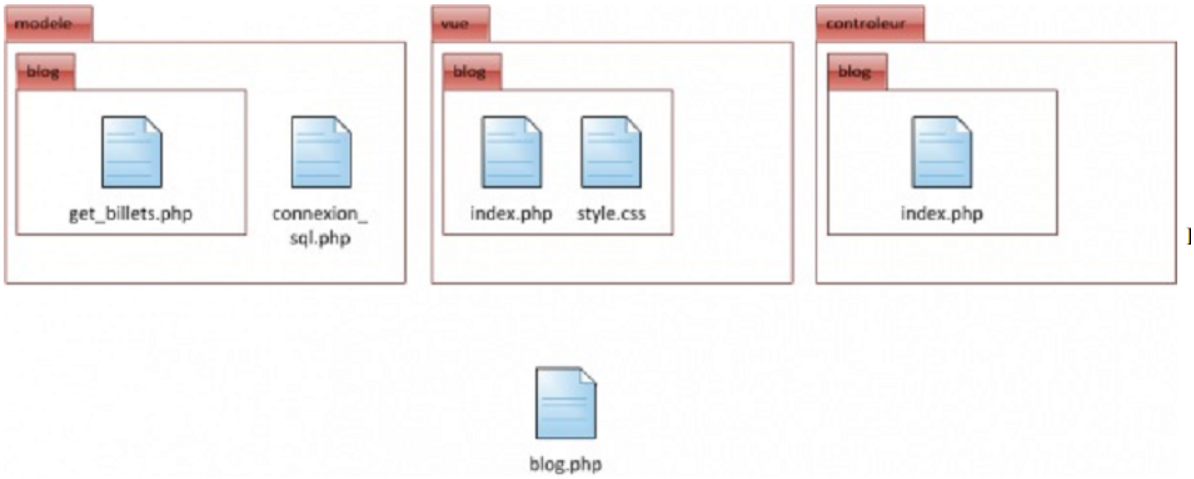
The controller only serves to make the connection between the model and the view.

In the simplest case, this will probably be the case. But as I said, the role of the controller is not limited to this: if there are calculations or permission checks to be done to miniaturize images, it is it who is in charge.

Concretely, the visitor will ask the page to the controller and it is the view which will be returned, as shown schematically in the following figure. Of course, this is all transparent to him, he does not see everything that happens on the server. This is the pattern you have been used, of course: it is however on this type of architecture that a large number of professional websites based!



PRACTICAL:



CONVERT MINICHAT

Top 10 Most Usable Content Management Systems

There are plenty of options when it comes to picking a content management system for a development project. Depending on how advanced you need the CMS to be, what language it's built in, and who is going to be using it, it can be a nightmare trying to find the "perfect" CMS for a project.

However, some CMSs have a slight edge over the rest of the competition because of the usability of the software. Some are just easier to install, use and extend, thanks to some thoughtful planning by the lead developers.

We have a number of themes and resources to support these top content management systems. If you're looking for Wordpress Themes, Drupal Themes, or Joomla Themes we have you covered on Envato Market.

We support a number of additional popular CMS systems. And a gallery of Wordpress Plugins, Drupal Plugins, Joomla plugins and more. Visit our ThemeForest or CodeCanyon marketplaces to browse through a ton of professional options.

Here are 10 of the most usable CMSs on the web to use in your next project, so you can choose the one that fits your needs best.

1. WordPress

WordPress
What is there left to say about WordPress that hasn't already been said? The PHP blogging platform is far and away the most popular CMS for blogging, and probably the most popular CMS overall. It's a great platform for beginners, thanks to their excellent documentation

and super-quick installation wizard. Five minutes to a running CMS is pretty good. Not to mention the fact that the newest versions auto-update the core and plugins from within the backend, without having to download a single file.

For those users not familiar with HTML or other markup language, a WYSIWYG editor is provided straight out of the box. The backend layout is streamlined and intuitive, and a new user should be able to easily find their way around the administration section. Wordpres also comes with built-in image and multimedia uploading support.

For developers, the theming language is fairly simple and straightforward, as well the Plugin API.

The WordPress Community is a faithful and zealous bunch. Wordpress probably has the widest base of plugins and themes to choose from. We have thousands of professional Wordpress Themes and Wordpress Plugins available for sale on Envato Market, with a full suite of styles and options to choose from.

A great part about the Wordpress community is the amount of help and documentation online you can find on nearly every aspect of customizing WordPress. If you can dream it, chances are it's already been done with WordPress and documented somewhere.

If you need help with anything from installing a theme to optimizing the speed of your WordPress site, you can find plenty of experienced WordPress developers to help you on Envato Studio.

2. Drupal

Drupal

Drupal is another CMS that has a very large, active community. Instead of focusing on blogging as a platform, Drupal is more of a pure CMS. A plain installation comes with a ton of optional modules that can add lots of interesting features like forums, user blogs, OpenID, profiles and more. It's trivial to create a site with social features with a simple install of Drupal. In fact, with a few 3rd party modules you can create some interesting site clones with little effort.

One of Drupal's most popular features is the Taxonomy module, a feature that allows for multiple levels and types of categories for content types. And you can find plenty of professional Drupal Themes, which are ready to be customized and worked with. You can also grab Drupal Plugins.

Drupal also has a very active community powering it, and has excellent support for plugins and other general questions.

You can also hire a developer to complete a range of tasks for your Drupal site for a reasonable fixed fee.

3. Joomla!

Joomla

Joomla is a very advanced CMS in terms of functionality. That said, getting started with Joomla is fairly easy, thanks to Joomla's installer. Joomla's installer is meant to work on common shared hosting packages, and is a very straightforward considering how configurable the software is.

Joomla is very similar to Drupal in that it's a complete CMS, and might be a bit much for a simple portfolio site. It comes with an attractive administration interface, complete with intuitive drop-down menus and other features. The CMS also has great support for access control protocols like LDAP, OpenID and even Gmail.com.

The Joomla site hosts more than 3,200 extensions, so you know the developer community behind the popular CMS is alive and kicking. Like Wordpress, you can add just about any needed functionality with an extension. However, the Joomla theme and extension community relies more on paid resources, so if you're looking for customizations, be ready to pull out your wallet. You can also grab Joomla plugins, or hire Joomla developers to help you get your store set up right.

4. ExpressionEngine

EE

ExpressionEngine (EE) is an elegant, flexible CMS solution for any type of project. Designed to be extensible and easy to modify, EE sets itself apart in how clean and intuitive their user administration area is. It takes only a matter of minutes to understand the layout of the backend and to start creating content or modify the look. It's fantastic for creating websites for less-than-savvy clients that need to use the backend without getting confused.

ExpressionEngine is packed with helpful features like the ability to have multiple sites with one installation of software. For designers, EE has a powerful templating engine that has custom global variables, custom SQL queries and a built in versioning system. Template caching, query caching and tag caching keep the site running quickly too.

One of my favorite features of EE that is the global search and replace functionality. Anyone who's ever managed a site or blog knows how useful it is to change lots of data without having to manually search and open each page or post to modify it.

ExpressionEngine is quite different than other previously-mentioned CMS in that it's paid software. The personal license costs $99.95, and the commercial license costs $249.99. You can also get help with ExpressionEngine on Envato Studio.

5. TextPattern

Textpattern

Textpattern is a popular choice for designers because of its simple elegance. Textpattern isn't a CMS that throws in every feature it can think of. The code base is svelte and minimal. The main goal of Textpattern is to provide an excellent CMS that creates well-structured, standards-compliant pages. Instead of providing a WYSIWYG editor, Textpattern uses textile markup in the textareas to create HTML elements within the pages. The pages that are generated are extremely lightweight and fast-loading.

Even though Textpattern is deliberately simple in design, the backend is surprisingly easy to use and intuitive. New users should be able to find their way around the administration section easily.

While Textpattern may be very minimal at the core level, you can always extend the functionality by 3rd party extensions, mods or plugins. Textpattern has an active developer community with lots of help and resources at their Textpattern.org site.

6. Radiant CMS

Radiant

The content management systems that we've listed so far are all PHP programs. PHP is the most popular language for web development, but that doesn't mean we should overlook other popular web languages like Ruby. Radiant CMS is a fast, minimal CMS that might be compared to Textpattern. Radiant is built on the popular Ruby framework Rails, and the developers behind Radiant have done their best to make the software as simple and elegant as possible, with just the right amount of functionality. Like Textpattern, Radiant doesn't come with a WYSIWYG editor and relies on Textile markup to create rich HTML. Radiant also has it's own templating language Radius which is very similar to HTML for intuitive template creation.

7. Cushy CMS

Cushy CMS

Cushy CMS is a different type of CMS altogether. Sure, it has all the basic functionality of a regular content management system, but it doesn't rely on a specific language. In fact, the CMS is a hosted solution. There are no downloads or future upgrades to worry about.

How Cushy works is it takes FTP info and uploads content on to the server, which in turn the developer or the designer can modify the layout, as well as the posting fields in the backend, just by changing the style classes of the styles. Very, very simple.

Cushy CMS is free for anyone, even for professional use. There is an option to upgrade to a pro account to use your own logo and color scheme, as well as other fine-grain customizations in the way Cushy CMS functions.

8. SilverStripe

SilverStripe

SilverStripe is another PHP CMS that behaves much like Wordpress, except has many more configurable options and is tailored towards content management, and not blogging. SilverStripe

is unique because it was built upon its very own PHP framework Saphire. It also provides its own templating language to help with the design process.

SilverStripe also has some interesting features built in to the base, like content version control and native SEO support. What's really unique with SilverStripe is that developers and designers can customize the administration area for their clients, if need be. While the development community isn't as large as other projects there are some modules, themes and widgets to add functionality. Also, you'll want to modify the theme for each site, as SilverStripe doesn't provide much in terms of style, to give the designer more freedom.

9. Alfresco

Alfresco

Alfresco is a JSP is a beefy enterprise content management solution that is surprisingly easy to install. A really useful feature of Alfresco is the ability to drop files into folders and turn them into web documents. Alfresco might be a little bit more work than some of the other CMS and isn't as beginner-friendly, it certainly is quite usable given the massive power of the system. The administration backend is clean and well-designed.

While Alfresco might not be a great choice for most simple sites, it's an excellent choice for enterprise needs.

10. TYPOlight

TYPOlight

TYPOlight seems to have the perfect balance of features built into the CMS. In terms of functionality, TYPOlight ranks with Drupal and ExpressionEngine, and even offers some unique bundled modules like newsletters and calendars. Developers can save time with the built-in CSS generator, and there are plenty of resources for learning more about the CMS.

If there is a downside to TYPOlight, it's that it has so many features and configurable options. Even though the backend is thoughtfully organized, there are still a lot of options to consider. But if you're wanting to build a site with advanced functionality and little extra programming, TYPOlight could be a great fit.

REFERENCES

[1] WIKIPEDIA encyclopedia http://en.wikipedia.org/wiki/PHP 21/01/2013
[2] Svein Nordbotten Introduction to PHP5 with MySQL Svein Nordbotten & Associates Bergen 2009
[3] Variables http://www.tutorialspoint.com/php/php_variable_types.htm 29/05/2013
[4] Form type http://www.w3schools.com/html/html_forms.asp